“Library Mobile App for Member Self Service”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

Sri Charan Kuchuru	**2520030093**
Rishi Alapati	**2420030268**
P. Sudha Gayathri	**2520030455**

Under the Esteemed Guidance of

Dr Spandana . S
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



Declaration

The Project Report entitled **"Library Mobile App for Member Self Service"** is a record of Bonafide work of **Sri Charan Kuchuru - 2520030093 , Rishi Alapati- 2520030268, P. Sudha Gayathri- 2520030455** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Sri Charan Kuchuru - 2520030093

Rishi Alapati - 2520030268

P. Sudha Gayathri - 2520030455

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is certify that the mini project based report entitled **"Library Mobile App for Member Self Service"** is a bonafide work done and submitted by **Sri Charan Kuchuru - 2520030093 , Rishi Alapati - 2520030268, P. Sudha Gayathri – 2520030455** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr Ramatenki Sateesh Kumar, and my guide Dr **Spandana . S** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)** Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Sri Charan Kuchuru – 2520030093

Rishi Alapati – 2520030268

P. Sudha Gayathri – 2520030455

Abstract

The Library Mobile App for Member Self-Service is a mobile-based library management solution designed to make library services faster, easier, and more accessible to students and other library members. In traditional library systems, members often need to visit the library or depend on staff for tasks such as searching for books, checking availability, viewing borrowing details, renewing books, and monitoring due dates. This can result in long waiting times, manual work, and difficulties in managing library resources efficiently.

The proposed application provides a self-service digital platform through which members can access essential library services using their mobile devices. Users can securely log in, search and browse the library catalog, view book details and availability, check their issued books and due dates, request or reserve books, and renew eligible books. The system can also provide notifications for upcoming due dates, overdue books, reservations, and other library updates. A backend database stores and manages member, book, borrowing, reservation, and transaction information while maintaining data consistency and security.

The project applies Relational Database Engineering concepts by designing an Entity-Relationship model and normalized relational schema using MySQL. SQL features such as DDL, DML, joins, subqueries, aggregates, constraints, indexes, views, transactions, and stored logic can be used to maintain data integrity and support efficient operations. The backend is developed using FastAPI, with Pydantic for validation, SQLAlchemy for database integration, and JWT-based authentication for security.

The system follows RESTful API principles and a layered backend architecture separating routers, services, and repositories. Docker and Docker Compose can be used for containerization, while GitHub supports version control and collaboration. API documentation and testing are performed using OpenAPI/Swagger and Postman, with Pytest used for backend testing. Overall, the project demonstrates database engineering, secure backend API development, and deployment practices while delivering a practical and user-friendly library self-service solution.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	5-15
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	16-21
3	Technology Stack	3.1 Front-end (e.g., React.js, Angular, HTML/CSS) 3.2 Back-end (e.g., Node.js, Django, Spring Boot) 3.3 Database (e.g., MongoDB, MySQL, PostgreSQL) 3.4 Version Control (e.g., Git, GitHub) 3.5 APIs or External services (e.g., Firebase, Stripe)	22-32
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)	33-36
5	Design	5.2 ER Diagram / Database schema	37
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	38-47
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	48-55
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	56-64
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	65-73
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	74-82
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	83-88
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	89-97
13	References	- Books, tutorials, APIs, documentation sites used	98-99

14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide -Geo Tag photos with guide -Review forms with guide signatures	100-105

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Libraries have long been central to learning, research, and community knowledge-sharing. Traditionally, they relied on manual registers, index cards, and physical catalogues to track books, members, and transactions. A member who wanted to borrow a book had to visit the library, search the shelves or consult a librarian, and complete the issue process at the counter. Returns, renewals, and fine payments followed the same in-person routine.

As libraries grew in collection size and membership, manual and semi-manual systems became hard to sustain. Records could be misplaced, duplicated, or entered incorrectly. Staff spent much of their time on repetitive tasks such as checking availability, calculating fines, and updating records, which left less time for helping members. Members, in turn, faced long queues and had no reliable way of knowing whether a book was available before travelling to the library.

Database Management Systems (DBMS) offer a structured answer to these problems. A relational database can store information about books, members, issue and return transactions, reservations, and fines in organised, interlinked tables. With a well-designed schema, data can be retrieved, updated, and validated quickly while keeping it consistent, secure, and free of unnecessary redundancy. Many libraries have adopted desktop-based library management software, but most of these tools are designed for staff, not for members.

Meanwhile, smartphones have become a part of everyday life. Students, researchers, and readers are used to handling banking, shopping, travel, and food delivery through mobile apps. They now expect the same convenience from library services. A mobile app lets members interact directly with the library's database from anywhere, at any time, without depending on staff for routine tasks.

This project, the Library Mobile App for Member Self-Service, responds to that need. It combines a mobile front end with a relational database back end so that members can search the catalogue, check availability, reserve books, renew loans, view their borrowing history, and track fines on their own. The project applies core DBMS concepts such as entity-relationship modelling, normalisation, SQL queries, constraints, and transaction management to a practical, real-world problem.

1.2 Problem Statement

Many libraries still depend on counter-based, staff-driven processes for routine member services. Even when a digital catalogue exists, it is often available only on library computers or in desktop software meant for staff. Members cannot easily check whether a book is available, when it is due back, or how much they owe without visiting the library or contacting a librarian. This leads to wasted time, unnecessary trips, and frustration.

The lack of self-service options also creates operational problems for the library. Librarians are overloaded with repetitive queries such as "Is this book available?", "When is my due date?", and "Can I renew this?". These interruptions slow down core work such as cataloguing, acquisitions, and member assistance. At busy times, such as exam periods, queues at the issue and return desk grow long and service quality suffers.

Data management is another significant concern. When records are kept in spreadsheets, paper registers, or loosely structured files, problems like data redundancy, inconsistency, and update anomalies appear. A book may be recorded as available when it has already been issued, or a member's fine may be calculated incorrectly. Without proper constraints and relationships, it is hard to maintain data integrity or to produce accurate reports about borrowing trends, overdue items, and member activity.

Members also lack transparency and control over their own accounts. They cannot see a clear history of the books they have borrowed, receive timely reminders about due dates, or reserve a title that is currently checked out. As a result, overdue returns and avoidable fines are common, and popular books are not distributed fairly among members who want them.

There is therefore a clear need for a secure, user-friendly mobile application, backed by a well-structured relational database, that allows members to perform key library tasks independently. The problem this project addresses is the absence of an integrated, mobile, self-service system that ensures accurate data, reduces dependence on staff, and gives members real-time access to library services.

1.3 Scope of the Project

The scope of this project covers the design and development of a mobile application for library members, supported by a relational database that stores and manages all library data. The focus is on member-facing self-service functions, with the database designed so that it can also support administrative operations performed by library staff.

On the member side, the application will include registration and secure login, profile viewing and updating, and catalogue search by title, author, subject, ISBN, or publisher. Members will be able to view detailed book information, including availability status and shelf location. They can reserve books that are currently issued to others, renew eligible loans, view current and past borrowings, check due dates, and see fines and payment status. Notifications or reminders for due dates and reservation availability may also be included.

On the database side, the project includes requirements analysis, entity-relationship (ER) design, conversion to a relational schema, and normalisation up to at least Third Normal Form (3NF). Core tables are expected to cover Members, Books, Authors, Categories, Issue/Return Transactions, Reservations, and Fines. Primary keys, foreign keys, and integrity constraints will be used to maintain consistency, and SQL queries, views, and, where appropriate, triggers and stored procedures will implement the application's logic.

Certain functions fall outside the scope of this project. These include full-scale acquisition and procurement management, integration with external library networks or inter-library loans, digital lending of e-books or other licensed content, RFID-based automated checkout hardware, and advanced analytics or recommendation engines. Online payment gateway integration may be simulated or limited to recording payment status rather than processing real transactions.

The application is intended for a single library or institution, such as a college, school, or community library, and is designed to be scalable so that more members, books, and features can be added later. The project will be developed and tested as an academic prototype, with sample data used to demonstrate functionality, performance, and data integrity.

1.4 Objectives of the Project

The main objective of this project is to design and develop a mobile application that lets library members carry out routine library tasks on their own, supported by a well-structured and reliable relational database. The project aims to improve the efficiency, accessibility, and accuracy of library services while demonstrating practical application of DBMS principles.

The first set of objectives concerns the member experience. The application should let members search the catalogue easily and check book availability in real time. It should allow them to reserve unavailable books, renew loans within library rules, and view their complete borrowing history. It should also give members clear visibility of due dates and outstanding fines, so that they can manage their accounts and avoid penalties without visiting the library.

The second set of objectives concerns database design and implementation. The project aims to analyse the library's data requirements and model them through an Entity-Relationship diagram. This model will then be converted into a normalised relational schema that reduces redundancy and prevents insertion, update, and deletion anomalies. The database will use primary keys, foreign keys, unique constraints, and check constraints to maintain data integrity and consistency across all tables.

The third set of objectives concerns functionality and security. The project aims to implement SQL queries, views, and, where suitable, stored procedures and triggers to automate tasks such as updating book availability after issue or return, calculating fines for overdue items, and enforcing borrowing limits. Secure authentication and role-based access will protect member data, so that each member can see and modify only their own information.

Finally, the project aims to reduce the workload on library staff by shifting repetitive enquiries and simple transactions to the app, freeing staff for more valuable work. It also aims to give the library accurate, up-to-date records that support better reporting and decision-making. Through testing with sample data, the project will verify that the system is correct, reliable, and easy to use.

1.5 Importance of the Application

The application is important because it brings library services to where members already spend their time: on their smartphones. Instead of travelling to the library just to check whether a book is available or to renew a loan, members can complete these tasks in seconds from anywhere. This saves time and effort, and it encourages more frequent and more informed use of library resources.

For members, the benefits go beyond convenience. Real-time visibility of due dates, reservations, and fines helps them stay organised and avoid late returns and penalties. The ability to reserve books and see availability ensures fairer access to popular titles. A personal borrowing history also helps students and researchers keep track of the resources they have used, supporting their academic work and reading habits.

For the library, the application improves operational efficiency. By automating routine enquiries and transactions, it reduces queues and the burden on staff, allowing librarians to focus on cataloguing, member support, and service improvement. Because the data are held in a well-designed relational database, records are accurate, consistent, and secure, and reports on circulation, overdue items, and popular titles can be generated quickly to support decisions about acquisitions and policies.

From an academic perspective, the project is important because it applies DBMS theory to a realistic problem. It brings together ER modelling, normalisation, SQL, integrity constraints, and transaction handling in one working system. This gives a clear demonstration of how sound database design underpins reliable software and shows how data-driven applications are built in practice.

More broadly, the application supports the modernisation and digital transformation of library services. It makes the library more accessible, particularly for members with limited time or mobility, and it moves the institution toward paperless, transparent, and user-centred operations. A scalable database foundation means the system can later grow to include notifications, e-resources, analytics, and other features as needs evolve.

1.6 Target Users / Audience

The primary users of the application are library members. In a college or university setting, these are mainly students, who borrow textbooks, reference materials, and reading material for coursework and exams. They typically own smartphones, are comfortable with mobile apps, and benefit most from quick searches, reservations, and renewal reminders during busy academic periods.

Faculty members and researchers form another important group. They often borrow specialised books, journals, and reference works, and may need to keep several items on loan for extended periods. For them, features such as easy renewals, borrowing history, and the ability to reserve in-demand titles save time and help them plan their research and teaching without frequent visits to the library.

In a school or public library, the audience also includes school students, general readers, and community members of different ages. The app should therefore have a simple, intuitive interface that is easy to use even for people with limited technical experience. Clear navigation, readable text, and straightforward workflows ensure that both tech-savvy and first-time users can use the self-service features comfortably.

The secondary audience is library staff, including librarians and administrators. Although the app is member-facing, staff benefit indirectly from the reduced workload and from the accurate, centralised data in the underlying database. With appropriate access controls, staff can use the same database to manage inventory, monitor circulation, verify fines, and generate reports, which makes their daily operations smoother and better informed.

Finally, the project is relevant to academic evaluators, instructors, and fellow students of database systems, who may examine it as a case study in database design and application development. For this audience, the project demonstrates how requirements are translated into a normalised schema, how SQL and constraints enforce correctness, and how a database connects to a mobile front end to deliver a practical service.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The Library Mobile App for Member Self-Service needs hardware at three levels: the development machine used to build the system, the server that hosts the database and application logic, and the mobile devices used by library members. Specifying each level helps ensure the system runs smoothly during development, testing, and real use.

For development, a laptop or desktop computer with at least an Intel Core i5 processor (or an AMD Ryzen 5 equivalent) is recommended. Building mobile apps, running emulators, and operating a database server together demand considerable processing power. A multi-core processor shortens build times and keeps the development environment responsive while several tools run at once.

The development machine should have a minimum of 8 GB of RAM, though 16 GB is preferable. Android emulators, code editors, database management tools, and web browsers all consume memory at the same time. Insufficient RAM causes slowdowns, frequent freezing, and failed builds, which would hamper productivity during implementation and testing.

Storage is the next requirement. At least 256 GB of free disk space is advisable, preferably on a solid-state drive (SSD). An SSD speeds up project loading, compilation, and database operations compared with a traditional hard disk. The space is needed for the operating system, development tools, SDKs, emulator images, project files, database backups, and documentation.

A stable internet connection is essential during development. It is required to download SDKs, libraries, and dependencies, to update tools, and to test communication between the mobile app and the server. A broadband connection of at least 10 Mbps is recommended, along with a standard display of 1366 × 768 resolution or higher so that code and design interfaces can be viewed comfortably.

For the server side, the prototype can run on a modest machine with a dual-core processor, 4 GB of RAM, and 50 GB of storage. This is enough to host the MySQL database and a lightweight

application server for a small number of concurrent users. In a real deployment, a cloud-hosted virtual machine or a dedicated institutional server with higher specifications would be chosen to support more members and larger data volumes.

The server should also provide reliable network connectivity and a stable power supply, ideally with an uninterruptible power supply (UPS) when hosted on premises. Because the database holds member records, transaction history, and fine details, any interruption could cause data inconsistency or loss. Regular backup storage, such as an external drive or cloud backup, should accompany the server to protect against hardware failure.

On the client side, members need a smartphone running Android 8.0 (Oreo) or later, or an iPhone running iOS 12 or later if the app is built for both platforms. These versions cover the great majority of devices in use and support the modern APIs the application relies on. The device should have at least 2 GB of RAM and a reasonably current processor to run the app without lag.

The mobile device should have at least 100 MB of free storage for installing the app and caching data such as search results and profile information. A display of 5 inches or more is recommended so that the catalogue, book details, and account screens are readable and easy to navigate by touch. The interface will be designed to adapt to different screen sizes and resolutions.

Connectivity is the final client-side requirement. Members need Wi-Fi or a mobile data connection (3G or better, with 4G preferred) because the app retrieves live availability, reservations, and fine information from the central database. Optional hardware features, such as a camera for scanning book barcodes or QR codes and notification support, can enhance the experience but are not mandatory for core functions.

2.2 Software Requirements

Software requirements describe the operating systems, runtimes, database, and supporting software needed to develop, deploy, and run the application. The chosen software is open source, widely supported, and well suited to building a database-driven web application that works well on mobile screens.

The development environment requires a modern operating system such as Windows 10 or 11, macOS, or a recent Linux distribution such as Ubuntu. All the tools used in the project run on these platforms, so team members can work on whichever system they have, and the application behaves the same on each.

The core of the system is the relational database management system. MySQL 8.0 or later is used because it is open source, reliable, and well documented. It supports SQL standards, primary and foreign keys, indexing, views, stored procedures, and triggers, all central to the library database design. It also integrates smoothly with Spring Boot through JDBC drivers.

The back end requires the Java Development Kit (JDK), version 17 or later. Spring Boot runs on the Java Virtual Machine, and recent long-term support releases of the JDK offer good performance and security. Apache Maven or Gradle is used as the build tool to manage dependencies, compile the code, run tests, and package the application as a runnable JAR file.

The back-end framework is Spring Boot, together with Spring Web for REST controllers, Spring Data JPA for database access, and Spring Security for authentication and authorisation. Spring Boot's embedded Tomcat server removes the need to install a separate web server, and its auto-configuration greatly reduces setup effort.

The front end requires Node.js (version 18 or later) and its package manager, npm. Node.js is used for the development tooling rather than as the production server: it runs the React development server, installs libraries, and builds the optimised production bundle. React is the front-end library, with HTML, CSS, and JavaScript as the underlying web technologies.

A graphical database tool is needed for design and administration. MySQL Workbench supports ER modelling, table creation, SQL execution, and user management. Alternatively, phpMyAdmin through XAMPP or a similar package provides a browser-based interface. These tools make schema design and data entry faster and reduce errors.

Testing software includes a modern browser, such as Google Chrome, with its developer tools and device emulation mode to check how the app behaves on different phone screen sizes. Postman is used to test the REST endpoints independently of the user interface, with sample requests for login, search, reservation, renewal, and fines. JUnit and Mockito, bundled with Spring Boot, support unit testing of the back end.

Version control software is also required. Git tracks the history of the code and database scripts, and a platform such as GitHub hosts the repository online for backup and collaboration. Without version control, it would be difficult to manage changes from multiple contributors or to recover from mistakes.

Finally, the project depends on security and documentation software. Spring Security with BCrypt password encoding and JSON Web Tokens (JWT) protects member credentials and sessions. Documentation and design tools such as Microsoft Word, draw.io, or Lucidchart are used to prepare the report, ER diagrams, data flow diagrams, and schema illustrations that accompany the working system.

2.3 Development Tools and Frameworks

The tools and frameworks chosen for this project determine how efficiently the system can be designed, coded, tested, and maintained. They were selected for their popularity, strong community support, free availability, and suitability for a three-tier application made of a React front end, a Spring Boot back end, and a MySQL database.

IntelliJ IDEA (Community or Ultimate edition) or Spring Tool Suite (STS) is used for the back end. These IDEs offer excellent support for Java and Spring, including code completion, dependency management, integrated debugging, and a built-in Maven or Gradle interface. Spring Initializr, available online and within the IDEs, is used to generate the initial project structure with the required dependencies.

Visual Studio Code is the main editor for the front end. It is lightweight and offers extensions for React, JavaScript, ESLint, and Prettier. Features such as syntax highlighting, IntelliSense, an integrated terminal, and live debugging support make it well suited to writing React components, HTML, and CSS.

React is the primary front-end library. Developed by Meta, it builds user interfaces from reusable components and uses a virtual DOM to update only the parts of a page that change. Hooks such as useState and useEffect manage component state and data fetching, and React Router handles navigation between pages without reloading the browser, giving an app-like experience.

The front end is bootstrapped with Vite or Create React App, which provide a ready-made build setup, a development server with hot reloading, and production bundling. For styling, plain CSS is combined with a responsive approach using Flexbox, CSS Grid, and media queries. A framework such as Bootstrap or Tailwind CSS can optionally speed up layout work and ensure consistency across screen sizes.

Axios or the built-in Fetch API is used to send HTTP requests from React to the back end. Axios provides convenient features such as request interceptors, which can attach the JWT token to every call automatically, and clear error handling. Centralising API calls in a service module keeps networking code separate from the user interface.

Spring Boot is the back-end framework, supported by several modules. Spring Web provides the REST controllers, Spring Data JPA with Hibernate maps Java classes to database tables, Spring Security handles authentication and authorisation, and Bean Validation checks incoming data. Lombok can be added to reduce boilerplate code such as getters and setters.

MySQL, managed through MySQL Workbench, is the main database tool. Workbench supports visual ER modelling, forward and reverse engineering of schemas, SQL script execution, and performance monitoring. Designing the database visually before implementation reduces mistakes in table relationships, and the generated scripts can recreate the database on any machine for testing or demonstration.

Postman is used to design and test the APIs, with saved collections documenting how each endpoint should behave. Git and GitHub provide version control, while Maven or Gradle manages builds and dependencies. Together they ensure that the back end can be verified before it is connected to the interface, and that the project can be built reliably on any system.

Design and documentation tools complete the toolset. Draw.io or Lucidchart are used for ER diagrams, data flow diagrams, use case diagrams, and flowcharts, while Figma can be used to sketch page layouts and user flows before coding begins. Microsoft Word is used to compile the final report. Together these tools give the project a structured, professional workflow from requirement gathering to final testing.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design

The front end of the Library Mobile App is everything the member sees and touches. Its main goal is to make library tasks such as searching, reserving, renewing, and checking fines quick, clear, and easy even for first-time users. The design follows a simple principle: every common action should take as few taps as possible.

The application is built with React, which uses a widget-based architecture. Every element on screen, from buttons and text fields to lists and cards, is a widget, and widgets are combined to form complete screens. This approach keeps the interface consistent and lets individual components, such as a book card, be reused across the app without rewriting code.

The app opens with a splash screen followed by a login and registration screen. New members register with details such as name, member ID, email, and password, while existing members log in with their credentials. Input fields include validation, so that empty fields, invalid email formats, and weak passwords are flagged immediately, before any request is sent to the server.

After login, members land on a home dashboard that gives a quick summary of their account. It shows the number of books currently borrowed, upcoming due dates, active reservations, and any outstanding fines. A prominent search bar sits at the top, with shortcuts to categories, new arrivals, and popular titles, so that the most frequent actions are always within reach.

The catalogue search screen lets members search by title, author, subject, ISBN, or publisher. Results are displayed as a scrollable list of cards, each showing the book's cover or icon, title, author, and an availability badge. Filters and sorting options help narrow large result sets, and tapping a card opens the book details screen.

The book details screen presents the full record: title, author, publisher, edition, category, shelf location, and the number of copies available. Depending on availability, a context-aware button appears, either "Reserve" for an issued title or "Reserved/Available" for others. This keeps the screen uncluttered while guiding the member towards the right action.

The My Account section groups the member's personal library information into tabs: Current Loans, Reservations, History, and Fines. In Current Loans, each item shows its due date and a

"Renew" button, which is disabled if the book is not eligible for renewal. Overdue items are highlighted in red, and fine amounts are displayed clearly so that members always know where they stand.

Navigation is handled by a bottom navigation bar with the main sections: Home, Search, My Books, and Profile. This pattern is familiar from popular apps, reducing the learning curve. A consistent colour scheme, readable typography, and generous spacing improve legibility, and the layout uses responsive widgets so that it adapts to different screen sizes and orientations.

Feedback and error handling are built into the design. Loading indicators appear while data is being fetched, success messages confirm completed actions such as a reservation, and friendly error messages explain problems like a lost connection or an invalid login. Confirmation dialogs appear before critical actions, so that members do not reserve or renew items by accident.

State management is handled using a pattern such as Provider, which keeps screen data synchronised with the information received from the server. The front end communicates with the back end only through HTTP requests, never directly with the database. Basic accessibility considerations, including adequate contrast, adjustable text, and clear labels, further ensure that the app is usable by a wide range of members.

3.2 Back-End Design

The back end is the logic layer between the React front end and the MySQL database. It receives requests, verifies who is making them, applies the library's rules, interacts with the database, and returns the results. Keeping this logic on a server centralises control and protects the database from direct access.

The back end is built with Spring Boot and follows a layered architecture. The controller layer defines REST endpoints and handles requests and responses, the service layer contains the business rules, the repository layer communicates with the database, and the entity layer represents the data model. Separating these responsibilities makes the code easier to read, test, and extend.

Communication follows the REST style over HTTP. Each resource, such as books, members, loans, reservations, and fines, has its own endpoints, using standard methods: GET to retrieve data, POST to create it, PUT to update it, and DELETE to remove it. Controllers are annotated

with @RestController and @RequestMapping, and data is exchanged in JSON, which React handles naturally.

Authentication is handled by Spring Security. When a member logs in, the credentials are checked against the stored data, where passwords are kept as BCrypt hashes rather than plain text. On success, the server issues a JSON Web Token (JWT) that the front end attaches to later requests, allowing the server to identify the member without asking for credentials each time.

Authorisation is enforced through security filters and role-based rules. A JWT filter runs before protected endpoints, verifies the token, and sets the authenticated user. Members can view and modify only their own loans, reservations, and profile, while administrative functions, such as adding books or recording returns, are restricted to users with a staff or admin role. CORS is configured so only the React front end may call the API.

The core business logic lives in the service layer. Examples include checking that a book has available copies before issuing it, enforcing a maximum number of borrowed books per member, refusing renewals on reserved or overdue items, and calculating fines at a fixed rate per overdue day. Placing these rules on the server guarantees they are applied uniformly, whatever the client.

Database access uses Spring Data JPA with Hibernate. Entity classes annotated with @Entity map to database tables, and repository interfaces extending JpaRepository provide ready-made methods for saving, finding, and deleting records. Custom queries are added with derived method names or the @Query annotation. Hibernate uses parameterised queries by default, which protects against SQL injection, and HikariCP provides efficient connection pooling.

Operations that change several tables must be handled as transactions. Issuing a book involves inserting a loan record and reducing the available copies, and both steps must succeed or fail together. Spring's @Transactional annotation wraps such operations so that they commit on success and roll back on failure, preserving atomicity and consistency.

Input validation and error handling ensure reliability. Request bodies are mapped to DTO classes validated with annotations such as @NotNull, @Email, and @Size. A global exception handler using @ControllerAdvice catches errors and returns meaningful HTTP status codes,

such as 400 for invalid input, 401 for unauthenticated access, 403 for forbidden actions, 404 for missing resources, and 500 for server errors, along with clear messages for the front end.

Finally, the back end addresses configuration and maintainability. Sensitive settings, such as the database credentials and the JWT secret, are stored in the application.properties file or environment variables instead of in code, with separate profiles for development and production. Logging records activity for debugging, and a scheduled task using @Scheduled can check for overdue items to update fines or trigger reminders. This structure keeps the server secure, observable, and ready to scale.

3.3 Database Design

The database is the foundation of the whole system, because every feature of the app depends on accurate, well-organised data. A relational model implemented in MySQL was chosen because the library domain naturally consists of related entities such as members, books, loans, and fines. The design followed requirements analysis, conceptual modelling, logical design, and physical implementation.

The conceptual design is represented by an Entity-Relationship (ER) diagram. The main entities are Member, Book, Author, Category, Publisher, Loan (issue and return), Reservation, and Fine. Each entity has attributes that describe it. For example, a Member has an ID, name, email, phone, password hash, membership date, and status, while a Book has an ISBN, title, edition, publisher, category, and number of copies.

The relationships between entities were carefully analysed. A member can borrow many books over time and a book can be borrowed by many members, a many-to-many relationship resolved through the Loan table. Similarly, a book can have several authors and an author can write several books, resolved through a Book_Author junction table. Each book belongs to one category and one publisher, while a category or publisher can have many books.

In the logical design, these entities were converted to relational tables. The Members table stores member details; the Books table stores catalogue information and copy counts; the Loans table records issue date, due date, return date, and status; the Reservations table records which member has reserved which book and when; and the Fines table records the amount, reason, and payment status linked to a loan.

Primary keys uniquely identify each row in every table, such as member_id, book_id, and loan_id, generated automatically using AUTO_INCREMENT. Foreign keys link related tables, for example loans.member_id references members.member_id and loans.book_id references books.book_id. These constraints enforce referential integrity, ensuring that no loan can exist for a member or book that is absent from the database.

The schema was normalised to Third Normal Form (3NF) to reduce redundancy and avoid update, insertion, and deletion anomalies. Repeating groups, such as multiple authors in a single field, were eliminated, partial dependencies were removed, and non-key attributes were made dependent only on the primary key. For example, category names are stored once in the Category table and referenced by ID.

Additional constraints protect data quality. NOT NULL constraints ensure essential fields are always filled, UNIQUE constraints prevent duplicate emails or ISBNs, and CHECK constraints keep values valid, for instance preventing negative copy counts or a due date earlier than the issue date. Default values, such as setting a loan status to "Issued" or a fine status to "Unpaid", simplify data entry and keep records consistent.

In the application, the tables are mapped to Java entity classes using Spring Data JPA and Hibernate. Annotations such as @Entity, @Table, @Id, @ManyToOne, and @ManyToMany describe the columns and relationships, so the object model mirrors the relational model. The schema can be created either from SQL scripts or generated through Hibernate's schema management, with scripts preferred for a controlled, reproducible structure.

Indexes improve query performance. Frequently searched columns, such as book title, author name, ISBN, and member email, are indexed so that catalogue searches return results quickly even as the collection grows. Views can be created for common queries, such as a list of overdue loans with member and book details, simplifying the queries used by the back end.

Finally, the database can include stored procedures and triggers for automation. A trigger can reduce available copies when a book is issued and increase them when it is returned, while another can calculate a fine automatically when a book is returned after its due date. Regular backups, restricted user privileges, and well-documented scripts complete the design, ensuring the database remains secure, recoverable, and easy to reproduce.

3.4 Version Control

Version control records changes to project files over time so that earlier versions can be recovered and different contributors can work together without overwriting one another. For this project, which includes React code, Spring Boot code, and database scripts, it is essential for organising work and protecting against accidental loss.

Git was chosen as the version control system because it is free, fast, and the industry standard. It is a distributed system, meaning every developer holds a complete copy of the repository and its history on their own machine. This allows work to continue offline, provides built-in redundancy, and makes operations such as committing and branching quick.

The project is hosted on GitHub, a cloud platform built around Git. A remote repository acts as the shared central copy, and developers synchronise their work using push and pull operations. GitHub also offers a web interface for browsing code, reviewing history, tracking issues, and managing contributions, which is convenient for both team work and evaluation by instructors.

The repository is organised into clear directories. A typical structure has a frontend folder containing the React application, a backend folder containing the Spring Boot project, and a database folder holding the schema creation script, sample data, triggers, and stored procedures. A README file explains the project, setup instructions, and how to run each part.

A simple branching strategy is followed. The main branch always holds stable, working code, while a develop branch collects ongoing work. For each new feature, such as login, catalogue search, or reservations, a separate feature branch is created. Isolating work this way means incomplete or experimental code never breaks the stable version of the project.

Commits are made frequently and carry short, descriptive messages, for example "Add book search endpoint" or "Fix fine calculation for overdue returns". Small, focused commits make it easy to understand what changed and why, and they allow a specific change to be undone without affecting unrelated work. This produces a readable history of how the project evolved.

When a feature is complete, its branch is merged into the develop branch, ideally through a pull request. A pull request lets teammates review the code, discuss improvements, and catch errors before the changes are integrated. If two people have edited the same part of a file, Git flags a merge conflict, which is then resolved manually to combine both contributions correctly.

A .gitignore file excludes files that should not be tracked, such as node_modules, the build and target folders, IDE settings, and, most importantly, configuration files containing passwords and secret keys. Keeping credentials such as the database password and JWT secret out of the repository is a basic security practice, particularly when the repository is hosted online.

Tags and releases mark important milestones, such as the completion of the database design or the first working prototype. If a new change introduces a bug, the team can compare versions, identify the commit responsible, and revert to a previous stable state. This safety net encourages experimentation, because mistakes are never permanent.

Version control also supports the academic side of the project. The commit history is evidence of steady progress and individual contribution, and the repository acts as a backup if a laptop fails. Database scripts stored alongside the code ensure that the exact structure used in the final demonstration can be recreated on any machine, making the project reproducible and professionally managed.

3.5 APIs or External Service

An Application Programming Interface (API) is a defined way for two software components to communicate. In this project, the most important API is the one built by the project itself: a REST API created with Spring Boot that lets the React front end talk to the back end and, through it, to the MySQL database. External services supplement it with features that would be costly to build from scratch.

The REST API exposes the library's functions as endpoints. Examples include POST /api/auth/login for authentication, GET /api/books for searching the catalogue, GET /api/books/{id} for details, POST /api/reservations to reserve a book, PUT /api/loans/{id}/renew to renew a loan, and GET /api/members/me/fines to view fines. Each endpoint has a clearly defined input and output, making the system predictable and easy to document.

All data is exchanged in JSON, and each response includes a suitable HTTP status code and, when needed, an error message. Protected endpoints require a valid JWT in the Authorization header. This design keeps the front end independent of the database, so the interface could be redesigned, or a native mobile app added later, without changing the back end.

Because the API is the core contract between the front end and the server, it is documented and tested carefully. Postman is used to build collections of requests and verify responses, and the API can be documented automatically with Swagger/OpenAPI through the springdoc-openapi library. This generates an interactive page where developers and evaluators can view and try each endpoint.

On the React side, Axios or the Fetch API is used to call the REST endpoints. Axios interceptors attach the JWT to each request and handle common errors, such as redirecting the user to the login page when a token expires. Centralising API calls in a dedicated service module keeps networking code separate from the components and makes later changes, such as altering the server address, simple.
Because the front end and back end run on different ports during development, Cross-Origin Resource Sharing (CORS) must be configured in Spring Boot to allow requests from the React application. In production, the allowed origins are restricted to the deployed front-end address. Proper CORS and HTTPS settings protect the API from unauthorised use by other websites.

Email is the first suggested external service. Using Spring Boot's JavaMailSender with an SMTP provider such as Gmail, the system can send registration confirmations, password reset links, and overdue notices. Email gives members a written record of important account events and acts as a reliable way to reach them even when they are not using the app.

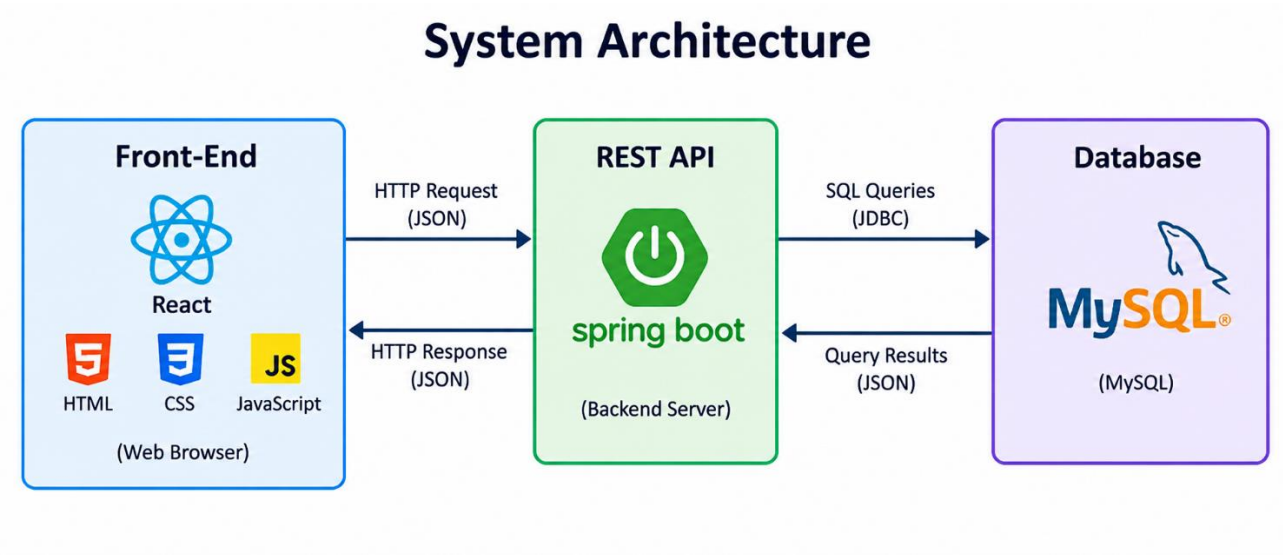
For book information, an external API such as the Google Books API or Open Library API can be integrated. By supplying an ISBN, the back end can retrieve cover images, descriptions, and other metadata using Spring's RestTemplate or WebClient. This enriches the catalogue and reduces manual data entry for library staff. These services are free within reasonable limits and return data in JSON, so they fit easily into the existing back end.

If fine payment is to be demonstrated, a payment gateway such as Razorpay or Stripe could be used in test mode. These providers supply secure checkout flows and sandbox environments, so payment can be simulated without handling real money or storing sensitive card details. In an academic prototype, simply recording the payment status in the Fines table is an acceptable alternative.

Using external services requires attention to security and reliability. API keys and secrets are stored in configuration files or environment variables, never in the React code or the public repository, and all communication takes place over HTTPS. The back end handles failures gracefully, for example by continuing to work if the email or book metadata service is temporarily unavailable. Because every external feature is optional, the core self-service functions remain independent of third-party availability.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram



4.2 Description of Each Layer (Frontend, Backend, Database)

The Library Mobile App for Member Self-Service follows a three-tier architecture made up of a presentation layer (frontend), an application layer (backend), and a data layer (database). Each layer has a distinct responsibility and communicates only with the layer next to it. Members interact with the frontend, the frontend talks to the backend through REST APIs, and only the backend talks to the database. This separation improves security, makes the system easier to maintain, and allows each layer to be changed or scaled independently.

The frontend is the presentation layer and is built with React, HTML, CSS, and JavaScript. It runs in the member's browser, whether on a phone, tablet, or computer, and is responsible for everything the member sees and touches. Its main duties are displaying the catalogue, search results, book details, loans, reservations, and fines, collecting input through forms, and giving instant feedback such as loading indicators and error messages.

Internally, the frontend is organised into reusable React components, such as the navigation bar, search box, book card, and loan list, grouped into pages like Login, Dashboard, Search, Book Details, My Books, and Profile. React Router moves the member between pages without reloading the browser, and component state manages data such as the current search results or the

logged-in member's details. CSS with Flexbox, Grid, and media queries makes the layout adapt to any screen size.

The frontend performs only light logic. It validates form input, such as checking that an email is correctly formatted or a password is long enough, to give quick feedback, but it never decides business rules like whether a book can be renewed. After login, it stores the JWT token received from the server and attaches it to every API request. Because the frontend is not trusted to enforce rules, every request is validated again on the server.

The backend is the application layer, built with Spring Boot and Java. It is the brain of the system: it receives requests from the frontend, authenticates the member, applies the library's business rules, communicates with the database, and sends back responses in JSON format. It is the only component allowed to access the database directly, which keeps sensitive data protected from direct exposure to users.

The backend itself is divided into sub-layers. Controllers expose the REST endpoints and handle incoming requests and outgoing responses. The service layer contains the business logic, such as checking availability before issuing a book, enforcing borrowing limits, allowing or refusing renewals, and calculating fines. The repository layer, built with Spring Data JPA, performs database operations, while entity classes represent the tables as Java objects.

Security is a central duty of the backend. Spring Security verifies login credentials against BCrypt-hashed passwords, issues and validates JWT tokens, and restricts access by role so that members see only their own data while staff can perform administrative tasks. The backend also handles validation, exception handling, transaction management, and logging, ensuring that the system responds consistently and recovers cleanly from errors.

The database is the data layer, implemented in MySQL. It is the permanent store of everything the system knows: members, books, authors, categories, publishers, loans, reservations, and fines. As a relational database, it organises this information into normalised tables connected by primary and foreign keys, which maintains consistency and prevents duplicated or contradictory data.

The database enforces data integrity independently of the application. Constraints such as NOT NULL, UNIQUE, CHECK, and foreign keys stop invalid data from being stored even if a bug exists in the code above it. Indexes on frequently searched columns, such as title, author, ISBN, and email, speed up queries, and optional views, triggers, and stored procedures automate tasks such as updating available copies and calculating fines.

The database layer also provides reliability and security. It supports transactions, so that multi-step operations like issuing a book either complete fully or not at all, and it uses restricted user accounts so that the application connects with only the privileges it needs. Regular backups protect the data against failure, and because only the backend can reach the database, members can never query it directly.

Together, the three layers form a clean flow. A member action, such as reserving a book, begins in the React interface, travels as an authenticated HTTP request to the Spring Boot controller, passes through the service layer's rules, is saved by the repository to MySQL, and returns as a JSON response that the interface displays. This layered design makes the system secure, testable, and scalable, and it lets future changes, like a new interface or a different database, affect only one layer.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

Deployment architecture describes where and how the application's components are installed and run so that members can use them. For this project, deployment is considered in three settings: a local environment used for development and demonstration, a cloud environment suitable for real-world use, and a CI/CD pipeline that automates the movement of code from the developer's machine to a running server.

In the local deployment, all three layers run on a single development machine. MySQL runs as a local database service on its default port 3306, the Spring Boot application runs on port 8080 using its embedded Tomcat server, and the React development server runs on port 3000 or 5173. The React app calls the Spring Boot API at localhost, with CORS enabled to allow the connection between the ports.

The local setup is used for coding, debugging, testing, and the final project demonstration. Its advantages are speed, zero hosting cost, and the ability to work without internet access. Hot reloading in React and automatic restarts in Spring Boot let developers see changes immediately. Environment-specific settings are kept in separate configuration profiles so that local database credentials never mix with production values.

For cloud deployment, the application is hosted on a platform such as Amazon Web Services (AWS), Microsoft Azure, Google Cloud, or a simpler platform like Render or Railway. A typical arrangement places each tier on its own service: the React build on a static hosting service, the Spring Boot application on a virtual machine or container service, and MySQL on a managed database service such as Amazon RDS. This keeps each layer independent and easier to scale.

The React frontend is compiled with the build command into optimised static files of HTML, CSS, and JavaScript. These files are uploaded to static hosting, such as Amazon S3 with CloudFront, Netlify, or Vercel, and served worldwide through a content delivery network (CDN). Because static files need no server-side processing, hosting is cheap and fast, and members receive pages quickly regardless of their location.

The Spring Boot backend is packaged as an executable JAR file using Maven or Gradle and deployed to a cloud server, for example an AWS EC2 instance, or run in a Docker container. Docker packages the application with the Java runtime and its dependencies, so it behaves identically on any machine. A reverse proxy such as Nginx can sit in front to handle HTTPS, forward requests to the application, and balance load if several instances run.

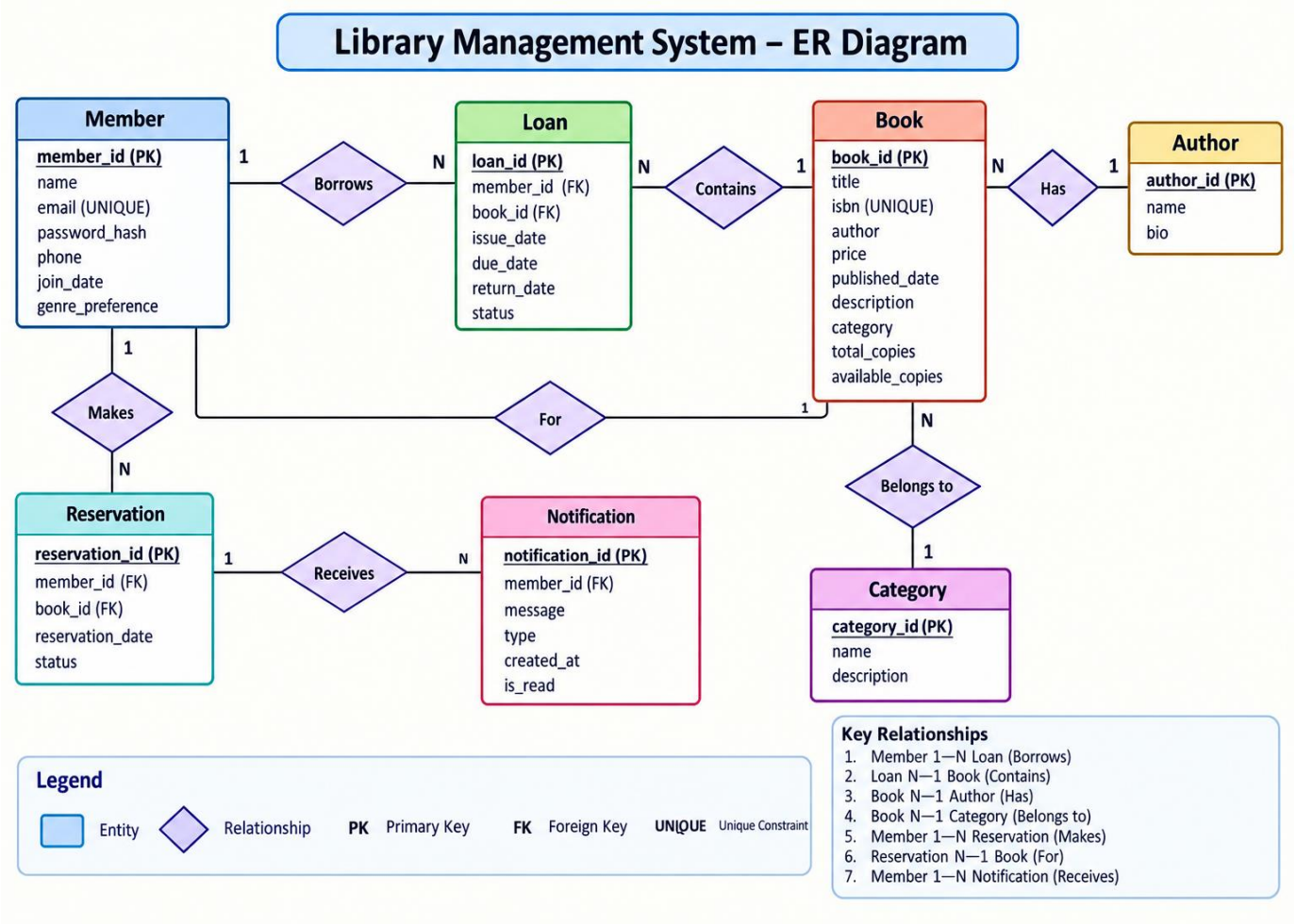
The managed MySQL database is placed in a private network that only the backend can reach, never exposed to the public internet. Automated daily backups, point-in-time recovery, and optional replicas for read-heavy workloads provide reliability. Security measures in the cloud include HTTPS certificates through a provider such as Let's Encrypt, firewalls and security groups that open only necessary ports, and environment variables or a secrets manager for database passwords and the JWT secret.

CI/CD stands for Continuous Integration and Continuous Delivery (or Deployment), and it automates building, testing, and releasing the software. The project's code is stored in GitHub, and a pipeline, defined with GitHub Actions in a YAML file, runs automatically whenever code is pushed or a pull request is opened. This removes manual, error-prone steps and ensures every change is checked in the same way.

The continuous integration stage runs first. The pipeline checks out the code, installs the Java and Node.js environments, builds the Spring Boot project with Maven or Gradle, and runs the JUnit tests. For the frontend, it installs dependencies with npm, runs linting and tests, and creates the production build. If any step fails, the pipeline stops and notifies the developers, so faulty code is caught before it reaches users.

The continuous delivery stage follows when the integration stage passes on the main branch. The pipeline builds a Docker image of the backend, pushes it to a container registry, and deploys it to the cloud server, while the React build is uploaded to the static host. Database changes are applied through versioned migration scripts, using a tool such as Flyway, so that the schema evolves safely alongside the code. After deployment, a health check confirms that the application is running, and the previous version can be restored quickly if a problem appears. This pipeline gives the project a fast, repeatable, and reliable path from code to production.

CHAPTER -5 DESIGN



CHAPTER -6 IMPLEMENTATION

6.1 Module-Wise Implementation

The Library Mobile App for Member Self-Service was implemented as a set of independent modules. Each module has its own React components, Spring Boot controller, service class, repository, and database tables. Building the system module by module let each feature be developed, tested, and integrated separately, and it kept the code organised as the project grew. The main modules are Member Management, Catalogue and Search, Loan Management, Reservation, Fine Management, Notification, and Administration.

The Member Management module handles registration, login, and profile maintenance. On the front end, a registration form collects the member's name, email, phone number, and password. On the back end, the AuthController receives the request, the service checks that the email is not already in use, hashes the password with BCrypt, and saves a new record in the Members table through the MemberRepository. The profile page lets a member view and update their details, and the update endpoint changes only the fields they are allowed to edit.

The Catalogue and Search module lets members find books. The BookController exposes endpoints to list books, search by title, author, subject, ISBN, or publisher, and fetch a single book's details. The BookRepository uses derived query methods and custom @Query statements with LIKE conditions for keyword search, and results support pagination so that large collections load quickly. The data is joined with the Author, Category, and Publisher tables so that each result shows complete information.

The React side of this module consists of a search bar, a filter panel, a result list of book cards, and a details page. As the member types or applies a filter, the component sends a request to the API and renders the returned list. A small delay (debouncing) before each request prevents the server from being flooded with calls on every keystroke. Each card shows an availability badge based on the number of copies available.

The Loan Management module covers issuing, returning, and renewing books. When a book is issued, the LoanService checks that the member's account is active, that the borrowing limit has not been reached, that the member has no unpaid fines above the allowed amount, and that at least one copy is available. If all checks pass, it creates a Loan record with the issue date and

a due date, and reduces the available copy count. These steps are wrapped in a @Transactional method so that either everything is saved or nothing is.

Returns and renewals follow similar logic. On return, the service records the return date, increases the available copies, and compares the return date with the due date; if the book is late, it triggers fine creation. For renewals, the service confirms that the loan is still active, has not been renewed beyond the permitted number of times, is not overdue, and has no pending reservation from another member. If so, the due date is extended. The My Books page shows each loan with its due date and a Renew button that is disabled when renewal is not allowed.

The Reservation module allows members to queue for a book that is currently unavailable. A reservation record stores the member, the book, the date, and a status such as Pending, Available, Cancelled, or Fulfilled. The service rejects duplicate reservations and prevents reservation of a book that is currently available. When a copy is returned, the system finds the earliest pending reservation for that book and changes its status to Available, so that the queue is served in first-come, first-served order.

The Fine Management module calculates and tracks penalties. When a late return is recorded, the FineService multiplies the number of overdue days by a fixed daily rate and creates a Fine record linked to the loan, with the status Unpaid. Members can view their fines in a dedicated tab, showing the amount, the book, the reason, and the payment status. Staff can mark a fine as paid after it is collected, and the member's ability to borrow can be restricted while large fines remain unpaid.

The Notification module keeps members informed. A scheduled job, annotated with @Scheduled, runs once a day and queries the database for loans due within the next few days and for loans already overdue. For each result it creates a notification entry and, optionally, sends an email through JavaMailSender. Members see their notifications in the app, such as reminders of approaching due dates and messages that a reserved book is ready, which reduces overdue returns.

The Administration module serves library staff. Protected endpoints allow authorised users to add, edit, or remove books, manage categories, authors, and publishers, register or deactivate

members, record returns, and collect fines. The React admin pages are available only to users with the staff role and use tables and forms for bulk data management. Basic reports, such as overdue items, most borrowed books, and active members, are produced from SQL queries and views.

Each module was verified separately before integration. The service classes were unit tested with JUnit and Mockito, the endpoints were tested with Postman using valid and invalid inputs, and the React components were checked in the browser using the device emulation mode. Integration testing then confirmed that the modules work together, for example, that a return updates the loan, the available copies, the fine, and the reservation queue in one consistent flow. This modular approach made debugging easier and left the system ready for new features.

6.2 Front-End Logic (UI Rendering and State Management)

The front-end logic of the application determines how data becomes visible screens and how the interface responds to the member's actions. Built with React, the front end uses a component-based approach in which the UI is described as a function of its state: when the underlying data changes, React automatically re-renders only the parts of the screen that depend on it. This keeps the interface consistent with the data and removes the need to update the page manually.

UI rendering starts from a root App component, which sets up routing and shared providers. React Router maps URLs to page components, such as /login, /dashboard, /search, /books/:id, /my-books, and /profile. Navigation between pages happens in the browser without a full reload, giving a smooth, app-like experience. Pages are composed of smaller reusable components, such as Navbar, SearchBar, BookCard, LoanItem, FineRow, and Alert, which receive their data through props.

Components are written as functions that return JSX, a syntax that combines HTML-like markup with JavaScript. Lists are rendered by mapping over arrays, so a list of books becomes a list of BookCard components, each with a unique key. Conditional rendering decides what appears based on data: an availability badge shows "Available" or "Reserve" depending on the copy count, an overdue loan is styled in red, and the admin menu appears only for staff users.

Local state is managed with the useState hook. A login form, for example, keeps the email and password in state, updating them on each keystroke so that the form always reflects what the member typed. Search pages keep the query text, selected filters, current page number, and result list in state. Because state updates trigger re-renders, the screen stays synchronised with the data without any manual manipulation of the page.

Side effects, such as fetching data from the server, are handled with the useEffect hook. When the Book Details page loads, an effect reads the book ID from the URL and requests the book's information from the API. A dependency array makes the effect run again only when needed, for example when the search query changes. Effects can also return cleanup functions, so pending requests are cancelled when the member leaves the page, avoiding errors and wasted work.

Global state is shared with the Context API. An AuthContext stores the logged-in member's details, the JWT token, and login and logout functions, making them available to any component without passing props through many levels. A similar context or the useReducer hook can hold values like the notification count. This avoids "prop drilling" while keeping the state management lightweight, which suits a project of this size without the added complexity of a library such as Redux.

Custom hooks encapsulate reusable logic. A useFetch hook, for instance, takes an endpoint, performs the request, and returns the data, a loading flag, and an error value. Components that use it can display a spinner while loading, an error message if the request fails, or the content when ready. This pattern keeps components short, removes duplicated code, and ensures consistent behaviour across all pages.

Form handling and validation are done on the client to give quick feedback. Each form checks its inputs before submission: required fields must be filled, emails must match a valid pattern, passwords must meet a minimum length, and dates must be logical. Error messages appear next to the relevant field, and the submit button is disabled while a request is in progress to prevent duplicate submissions. Client-side validation improves usability, while the server repeats the checks for security.

Protected routing controls which pages a member may view. A ProtectedRoute component checks the AuthContext: if no valid token exists, it redirects the visitor to the login page, and if the page requires the staff role, it verifies the role before rendering. When the member logs out or the token expires, the context clears the stored data and the member returns to the login screen, so private pages never remain visible.

Finally, the front end handles user feedback and responsiveness. Loading indicators, success banners, error alerts, and confirmation dialogs inform the member about the result of each action, such as a successful reservation or a failed renewal. CSS Flexbox, Grid, and media queries adjust layouts to the screen size, and rendering performance is improved by splitting pages into smaller components, using keys in lists, and loading routes lazily. Together these techniques make the interface fast, predictable, and pleasant to use on any device.

6.3 API Integration

API integration is the link between the React front end and the Spring Boot back end. All data used by the interface, such as books, loans, reservations, and fines, is obtained from RESTful endpoints, and all member actions are sent back through the same endpoints. The integration follows a clear request-response model in which the front end sends an HTTP request and the back end returns a JSON response with a status code.

The back end exposes a consistent set of endpoints grouped by resource. Examples include POST /api/auth/register and POST /api/auth/login for authentication, GET /api/books and GET /api/books/{id} for the catalogue, POST /api/loans/{id}/renew for renewals, POST /api/reservations and DELETE /api/reservations/{id} for reservations, and GET /api/members/me/loans and GET /api/members/me/fines for the member's account. Using nouns for resources and standard HTTP methods makes the API predictable and easy to learn.

On the front end, the API calls are centralised in a services folder rather than scattered through components. An Axios instance is created with the base URL of the back end, a timeout, and default headers. Separate service files, such as bookService, loanService, and authService, export functions like searchBooks, renewLoan, and getMyFines. Components simply call these functions, so a change to an endpoint needs updating in only one place.

Axios interceptors add important behaviour to every call. A request interceptor reads the JWT token from the authentication context or storage and attaches it to the Authorization header as a Bearer token. A response interceptor inspects the results: if the server returns 401 Unauthorized, indicating an expired or invalid token, it logs the member out and redirects them to the login page. This keeps security handling consistent without repeating code.

Requests are made asynchronously with async/await inside try-catch blocks. A typical flow sets a loading flag, awaits the response, stores the returned data in state, and then clears the loading flag. If an error occurs, the catch block extracts the message from the server's response and stores it in an error state so it can be displayed. This ensures the interface never freezes or fails silently when the network or server has a problem.

Data formats are agreed between the two sides through Data Transfer Objects (DTOs). In Spring Boot, request bodies are mapped to DTO classes such as LoginRequest, ReservationRequest, and BookResponse, and entities are converted to DTOs before being returned. This prevents sensitive fields, like password hashes, from leaking, and it lets the API shape stay stable even when the database structure changes. The React components expect the same JSON structure and use it directly for rendering.

Server-side validation and error responses are essential parts of the integration. Request DTOs are annotated with constraints such as @NotBlank, @Email, and @Size, and a global @ControllerAdvice class converts exceptions into a uniform JSON error with a status code, message, and timestamp. The front end reads these messages and displays them, for example "Book not available for renewal" or "Email already registered", so members understand exactly what went wrong.

Cross-Origin Resource Sharing (CORS) must be configured because the React app and the Spring Boot server run on different origins, particularly in development. The back end specifies the allowed origins, methods, and headers, including the Authorization header. In production, only the deployed front-end address is permitted. Proper CORS settings let the browser make legitimate calls while blocking requests from unknown websites.

For large result sets, the integration supports pagination and filtering. The search endpoint accepts query parameters such as keyword, category, page, and size, and Spring Data's Pageable interface returns a page of results along with the total count. The front end sends the parameters, displays one page at a time, and offers next and previous controls. This reduces the amount of data transferred and keeps the interface fast on mobile networks.

The integration was tested at several levels. Each endpoint was first tested in Postman with valid and invalid data to confirm the correct status codes and responses. Swagger/OpenAPI documentation generated by springdoc provides an interactive reference of all endpoints. Finally, end-to-end testing from the React interface confirmed that tokens, headers, request bodies, and error messages work together correctly. Environment variables hold the API base URL, so the same front-end code works with both the local and deployed servers.

6.4 Authentication and Authorisation

Authentication verifies who a user is, while authorisation determines what that user is allowed to do. Both are essential for the library app because it stores personal data and borrowing records, and because members and staff must have different levels of access. The system uses Spring Security on the back end with JSON Web Tokens (JWT), together with protected routes and context-based session handling on the React front end.

The process begins with registration. When a new member submits the registration form, the back end validates the data, checks that the email or member ID is not already registered, and hashes the password using the BCrypt algorithm before saving it. BCrypt adds a random salt and is deliberately slow, which makes brute-force and rainbow-table attacks impractical. The original password is never stored or logged, so even a database leak would not reveal it.

Login is handled by the authentication endpoint. The member submits an email and password, and Spring Security's AuthenticationManager loads the user through a UserDetailsService, which fetches the record from MySQL. The submitted password is compared with the stored hash using the BCrypt encoder. If the credentials do not match, the server returns a 401 response with a generic message, avoiding any hint about whether the email or the password was wrong.

When authentication succeeds, the server generates a JWT. A JWT is a compact, signed string made of three parts: a header, a payload, and a signature. The payload contains claims such as the member's ID or email, their role, the issue time, and an expiry time, typically a few hours. The token is signed with a secret key using an algorithm like HS256, so any tampering with its contents can be detected. The token is returned to the React app in the response.

The approach is stateless, meaning the server does not keep session data in memory. The front end stores the token, either in memory or in browser storage, and sends it in the Authorization header as "Bearer <token>" with every later request. Because the token carries the necessary identity information, any server instance can validate it independently, which makes the system easy to scale. Storing it in memory or in an HttpOnly cookie is more secure against script attacks than plain local storage.

On the back end, a custom JWT filter extending OncePerRequestFilter runs before each protected request. It reads the Authorization header, extracts the token, verifies the signature and expiry, and loads the corresponding user. If valid, it places an Authentication object in the SecurityContext, so the rest of the request is treated as coming from that user. If the token is missing, expired, or invalid, the request is rejected with a 401 Unauthorized response.

Authorisation is based on roles. Each user has a role such as ROLE_MEMBER or ROLE_ADMIN stored in the database and included in the token. In the SecurityFilterChain configuration, URL patterns are matched to roles: public endpoints like login and registration are open to everyone, member endpoints require authentication, and administrative endpoints under /api/admin require the admin role. Method-level annotations like @PreAuthorize("hasRole('ADMIN')") add a further layer of protection on sensitive operations.

Role checks alone are not enough, so the system also applies ownership checks. A member must be able to view and modify only their own loans, reservations, and fines. Instead of accepting a member ID from the request, the back end reads the identity from the authenticated token and queries only that member's data. This prevents a user from accessing another member's records simply by changing an ID in the URL, a flaw known as insecure direct object reference.

On the front end, the AuthContext holds the token, the member's details, and the login and logout functions. After login, the context stores the data and the router sends the member to the dashboard; on logout, it clears everything and returns to the login page. A ProtectedRoute component blocks access to private pages for unauthenticated visitors and hides admin pages from non-admin users. These checks improve usability, but the real enforcement always happens on the server.

Additional security measures complete the design. Passwords must meet minimum strength rules, CORS limits which websites may call the API, and HTTPS encrypts traffic between the browser and the server. Login attempts can be rate-limited to slow brute-force attacks, tokens have short lifetimes with an optional refresh-token mechanism, and the JWT secret is kept in environment variables rather than in the code. Together, these measures protect member data, keep staff functions restricted, and ensure that the system's access control is reliable.

CHAPTER 7- FEATURES

7.1 List of Main Features

The Library Mobile App for Member Self-Service provides a set of features that let members handle routine library tasks on their own, while giving staff the tools to manage the collection and the members. Each feature is described below. Section 7.2 explains how each one works internally.

Member Registration and Login. New members can create an account with their name, email, phone number, and password, and existing members can log in securely. The system verifies the credentials and gives access only to authenticated users. This feature protects personal data and links every action in the app to a specific member.

Profile Management. Members can view and update their personal details, such as name, phone number, and password. The profile page also shows the membership status and the date of joining. This lets members keep their records accurate without visiting the library or asking staff for changes.

Catalogue Search and Filtering. Members can search the library catalogue by title, author, subject, ISBN, or publisher. Filters for category and availability, along with sorting options, help narrow large result lists. This replaces the need to browse shelves or ask a librarian whether a book exists in the collection.

Book Details and Real-Time Availability. Each book has a details page showing its author, publisher, edition, category, shelf location, and the number of copies currently available. The availability status is always taken from the live database. Members can therefore decide whether to visit the library or reserve the book before travelling.

Book Reservation. When a book is currently issued to someone else, a member can reserve it and join the waiting queue. Members can view their active reservations and cancel them if they no longer need the book. This gives fair, first-come, first-served access to popular titles.

Loan Renewal. Members can extend the due date of a borrowed book directly from the app,

provided the renewal rules are met. The Renew button is enabled only for eligible loans. This saves a trip to the library and helps members avoid late fines.

Dashboard and Borrowing History. The home dashboard summarises the member's account, including books currently borrowed, upcoming due dates, active reservations, and outstanding fines. A history section lists all past loans and returns. Members can keep track of what they have read and what is due soon.

Fine Tracking. The system calculates fines automatically for books returned after their due date and displays them in a dedicated section. Each fine shows the amount, the book concerned, the reason, and whether it is paid. This makes penalties transparent and removes disputes about how much is owed.

Notifications and Reminders. Members receive reminders about approaching due dates, overdue items, and the availability of reserved books, within the app and optionally by email. These alerts reduce forgotten returns and make sure a reserved book is collected on time.

Administrative Management. Library staff have a protected area to add, edit, and remove books, manage categories, authors, and publishers, register or deactivate members, record returns, and collect fines. Staff can also view reports on overdue items and popular books. This keeps the catalogue and member records accurate and supports decision-making.

7.2 How Each Feature Works

Each feature of the application works through the same basic chain. The member acts in the React interface, the interface sends an authenticated request to a Spring Boot REST endpoint, the service layer applies the library's rules, the repository reads or updates the MySQL database, and a JSON response returns to update the screen. The paragraphs below explain this process for each feature in turn.

Member Registration and Login. On the registration page, React validates the form, then sends the details to the registration endpoint. The back end checks that the email is not already used, hashes the password with BCrypt, and saves the new member in the Members table. During login, Spring Security compares the submitted password with the stored hash. If it matches, the server creates a signed JWT containing the member's identity and role, and returns it. React

stores the token in the authentication context and attaches it to every later request.

Profile Management. When the profile page loads, an effect requests the current member's details using the token, so the server knows whose data to return without any ID being sent. The details are placed in form fields held in state. When the member saves changes, the form is validated and sent to the update endpoint, where the service checks the data and updates only the permitted fields. Password changes require the old password, and the new password is hashed before saving.

Catalogue Search and Filtering. As the member types a keyword or selects a filter, React waits briefly (debouncing) and then sends a request with query parameters such as keyword, category, page, and size. The BookRepository runs a query with LIKE conditions on the title, author, subject, ISBN, or publisher, joined with the related tables. Spring Data returns a page of results with the total count. React renders each result as a book card and provides next and previous controls for pagination.

Book Details and Real-Time Availability. Tapping a book card opens a page whose route contains the book ID. An effect reads the ID and calls the details endpoint, which returns the book's full information along with the number of available copies. Because the count is stored in the database and updated whenever a book is issued or returned, the figure is always current. React uses this number to show an Available or Reserve button, so the member sees only relevant actions.

Book Reservation. When the member taps Reserve, React sends the book ID to the reservation endpoint. The service confirms that the book is currently unavailable, that the member has not already reserved it, and that the member has not exceeded the reservation limit. It then saves a reservation with the status Pending and the current date. When another member returns the book, the return process finds the earliest pending reservation for that title and changes its status to Available, and a notification is created for that member.

Loan Renewal. On the My Books page, React fetches the member's active loans and decides whether to enable each Renew button. When the member taps it, a request goes to the renewal

endpoint, and the LoanService rechecks the rules on the server: the loan must be active, not overdue, below the maximum renewal count, and the book must have no pending reservation from someone else. If the checks pass, the due date is extended by the standard loan period and the renewal count increases. The updated due date is returned, and React refreshes the display.

Dashboard and Borrowing History. After login, the dashboard calls endpoints that return summary data for the authenticated member, such as the count of active loans, the nearest due date, the number of reservations, and the total unpaid fines. These figures come from aggregate queries, for example COUNT and SUM, on the Loans, Reservations, and Fines tables. The history page requests past loans in pages, sorted by date, and displays them in a list with issue and return dates, so the member can review their reading record.

Fine Tracking. Fines are created automatically when a book is returned. The FineService compares the return date with the due date, and if the book is late, it multiplies the number of overdue days by the daily rate and saves a Fine record linked to the loan with the status Unpaid. The Fines page requests the member's fines and shows the amount, book, reason, and status. When staff record a payment, the status changes to Paid, and borrowing restrictions linked to large unpaid balances are lifted.

Notifications and Reminders. A scheduled task in Spring Boot, marked with @Scheduled, runs once a day. It queries the database for loans due within the next few days, loans already overdue, and reservations whose status has become Available. For each match, it saves a notification for the relevant member and may send an email through JavaMailSender. When the member opens the app, React requests unread notifications and displays them with a count badge. Marking a notification as read updates its status in the database.

Administrative Management. Admin pages are reachable only by users whose token contains the admin role. React's protected route hides them from other users, and Spring Security enforces the restriction on the server through role-based rules on the admin endpoints. Staff submit forms to create or update books, authors, categories, and members, and the services validate and save them. When staff record a return, one transactional operation updates the loan, increases available copies, creates a fine if needed, and advances the reservation queue, keeping all the data consistent.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Postman was the main tool for testing the application's REST API. It lets each back-end endpoint be called directly, without the React interface, which isolates problems in the server logic from problems in the user interface. If a feature failed in the browser, the same request could be sent from Postman to find out whether the fault was in the front end, the back end, or the database.

Postman does not test the visual interface itself, so it was used for the back end and for the connection between the two layers. The front end was tested in the browser, and Postman confirmed that the data the front end sends and receives matches what the server expects. Together, the two approaches cover both sides of the system.

A Postman collection named "Library App API" was created, with folders for Authentication, Members, Books, Loans, Reservations, Fines, and Admin. Each folder contains saved requests with their HTTP method, URL, headers, and JSON body. Saving requests in a collection made tests repeatable and easy to rerun after each change to the code.

An environment was configured with variables such as baseUrl, token, and memberId. Using {{baseUrl}} in every request meant the same collection could be run against the local server and the deployed server by switching the environment. The token variable held the JWT, so authenticated requests did not need the token to be pasted by hand each time.

Authentication was tested first. A POST request to the login endpoint with valid credentials returned a 200 status and a JWT. A test script in Postman extracted the token from the response and stored it in the environment variable automatically. Later requests used it in the Authorization header as a Bearer token, so the whole collection could run in sequence after a single login.

Each endpoint was tested with both valid and invalid input. Valid requests were expected to return the correct status codes (200 for success, 201 for creation) and a response body in the correct format. Invalid requests, such as a missing field, a malformed email, a wrong password, or a non-existent book ID, were expected to return 400, 401, or 404 with a clear error message.

Security rules were also checked in Postman. Requests to protected endpoints without a token returned 401, and requests with a member token to admin endpoints returned 403. A member's token was used to try to read another member's loans, and the server correctly returned only the data belonging to the logged-in member. These checks confirmed that authentication and authorisation work as designed.

Postman's test scripts, written in JavaScript, were used to automate the checks. Scripts verified the status code, the presence of required fields in the response, the data types, and the response time. For example, a test could assert that the book search response returns an array and that each item contains a title and an availability count. Failed assertions were displayed immediately in the results panel.

The Collection Runner was used to execute all the requests in order, with test results summarised as passed or failed. Data files in CSV or JSON format allowed the same request to be run with many different inputs, such as several registrations or searches. This made it quick to run a full regression test after each change to confirm that nothing had broken.

Frontend and backend integration was checked by comparing Postman results with the browser. The browser's Network tab in developer tools showed the requests the React app actually sent, including headers and bodies, and these were copied into Postman for debugging when something failed. This approach quickly revealed problems such as missing Authorization headers, incorrect field names, and CORS errors, and made fixing them straightforward.

8.2 Integration Testing

Integration testing checks that the separate modules and layers of the application work correctly together. Unit tests show that each part works alone, but many defects only appear when parts interact, for example when the front end sends data in a format the back end does not expect, or when two modules update the same table. Integration testing was therefore carried out after the individual modules were completed.

The first level tested was the integration between the React front end and the Spring Boot back end. This confirmed that requests from the browser reach the correct endpoints, that JSON bodies are interpreted properly, that the JWT token is sent with each call, and that responses are displayed correctly. CORS configuration was verified at this stage, since a mistake there blocks

all front-end requests.

The second level tested the integration between the back end and the MySQL database. Here the aim was to confirm that entities map correctly to tables, that queries return the right data, and that constraints work as designed. For example, tests confirmed that saving a loan with a non-existent member ID is rejected by the foreign key, and that duplicate emails are rejected by the unique constraint.

The third level tested the integration between modules. The most important scenario is the book lifecycle, where several modules work in sequence. A member searches for a book, issues it, renews it, returns it late, and receives a fine. Each step was checked to confirm that the loan, available copies, fine record, and notification were all updated consistently across the related tables.

The reservation workflow was another key integration scenario. One member reserves a book that is unavailable, and another member returns it. The test verified that the available copy count increased, that the earliest pending reservation changed to Available, and that a notification was created for the correct member. This showed that the Loan, Reservation, and Notification modules cooperate correctly.

Transaction handling was tested explicitly. Operations such as issuing or returning a book modify several tables, and a failure in the middle must not leave the data half-updated. A forced error was introduced after the loan record was created but before the copy count was updated, and the test confirmed that the whole operation was rolled back, leaving the database unchanged.

Security integration was tested across the layers. The tests confirmed that a request without a token is blocked before reaching the controller, that an expired token causes the front end to log the member out and redirect to the login page, and that a member cannot see admin pages in the interface or call admin endpoints directly. This verified that front-end route protection and back-end enforcement agree.

Integration tests were also written in code using Spring Boot Test with @SpringBootTest and

MockMvc. These tests start the application context, send simulated HTTP requests to the controllers, and check the responses and the database state. A separate test database, or an in-memory database such as H2, was used so that tests did not affect real data. Running them automatically gave quick feedback after every change.

Test data was prepared before each scenario. Sample members, books, authors, and categories were inserted using SQL scripts or test setup methods, and cleaned up afterwards so that each test started from a known state. This made the results repeatable and prevented one test from affecting the result of another. Edge cases such as zero available copies, the maximum borrowing limit, and the last day before the due date were deliberately included.

Defects found during integration testing were recorded, fixed, and retested. Typical issues included a field name mismatch between the React form and the Spring DTO, a missing transaction annotation that allowed partial updates, and a date format difference between the browser and the server. After each fix, the affected scenarios and the Postman collection were run again to confirm that the fix worked and that no earlier functionality had broken.

8.3 Tools Used for Testing

A range of testing tools was used, each suited to a different layer of the application. Using purpose-built tools made testing faster, more reliable, and more repeatable than manual checking alone. The tools were chosen for being free, widely used, and compatible with the project's React, Spring Boot, and MySQL stack.

Postman was the main tool for API testing. It was used to send requests to every endpoint, check status codes and response bodies, and test authentication and authorisation rules. Its collections, environments, and test scripts allowed the whole API to be tested repeatedly, and the Collection Runner provided a quick summary of passes and failures after each code change.

JUnit 5 was used for unit testing the Java back end. It runs individual test methods and verifies expected results through assertions. The business rules in the service layer, such as fine calculation, borrowing limits, and renewal eligibility, were tested with JUnit using many input combinations. Because unit tests run in seconds, they could be executed after every change.

Mockito was used with JUnit to isolate the code being tested. It creates mock versions of dependencies, such as repositories, so that a service can be tested without a real database. For instance, a mocked BookRepository could be told to return a book with zero copies, allowing the test to confirm that the LoanService refuses to issue it. This kept unit tests fast and focused.

Spring Boot Test and MockMvc supported integration testing of the back end. They load the application context and simulate HTTP requests to controllers without needing a running server. Combined with Spring Security Test, they made it possible to simulate authenticated users with specific roles, which was essential for testing access control on member and admin endpoints.

For the front end, Jest and React Testing Library were used to test components. These tools render a component in a simulated environment, interact with it as a user would by typing, clicking, and submitting, and check what appears on the screen. They were used to verify form validation messages, conditional rendering such as the Reserve button, and the behaviour of the protected route component.

The Chrome browser and its Developer Tools were used for manual and responsive testing. The Network tab shows every request and response, the Console shows JavaScript errors, and device mode simulates different phone and tablet screen sizes. This was essential for confirming that the interface works well on mobile screens, which is a key requirement of the project.

MySQL Workbench was used to inspect and verify the database during testing. After running an operation such as issuing a book, a SQL query in Workbench confirmed that the loan was inserted, the available copies decreased, and the related records were correct. It was also used to prepare test data, reset tables between test runs, and check that indexes and constraints exist.

H2 in-memory database was used for automated back-end tests where a lightweight database was sufficient. It starts instantly, resets after each run, and requires no setup, which keeps tests independent of the real MySQL server. For final verification, however, key scenarios were repeated against MySQL to ensure that behaviour matched the real database engine.

Supporting tools included Git and GitHub for tracking test code and results, and GitHub Actions for running the automated tests on every push. IDE tools in IntelliJ IDEA or VS Code displayed

test results and supported debugging with breakpoints. Together, these tools gave coverage from individual methods up to complete user workflows, and kept the testing process organised and repeatable.

8.4 Test Cases and Results

Test cases were designed to cover the main features of the application, including normal use, invalid input, and security rules. Each test case has an ID, the module being tested, the input or action, the expected result, the actual result, and a pass or fail status. Both positive tests (valid input should work) and negative tests (invalid input should be rejected cleanly) were included.

The test cases were derived from the requirements. For each feature, such as registration, login, search, reservation, renewal, and fines, at least one successful scenario and one failure scenario were written. Boundary conditions were also tested, for example borrowing when the limit is exactly reached, renewing on the last allowed renewal, and returning a book on the exact due date. These cases often reveal errors that ordinary tests miss.

Each test was executed using Postman for the API, the browser for the user interface, and automated JUnit tests for business logic. The actual result was compared with the expected result. When they matched, the test was marked Pass. When they differed, the defect was logged, the code was corrected, and the test was repeated until it passed. The table below summarises the main test cases.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

Deployment is the process of moving the finished application from the developer's computer to a server where members can use it. For this project, deployment involves three parts: preparing the MySQL database, packaging and running the Spring Boot back end, and building and publishing the React front end. Following a fixed sequence of steps keeps the process repeatable and reduces the chance of errors.

The first step is to prepare the source code. All features are merged into the main branch of the GitHub repository, and the automated tests are run to confirm that nothing is broken. Debugging code, test accounts, and hard-coded passwords are removed, and the version is tagged in Git so that it can be recovered later. Deploying only tested, committed code ensures the live system matches a known version.

The second step is to set up the database server. MySQL 8.0 or later is installed on the host machine, or a managed MySQL instance is created on a cloud platform. A new database for the library is created with a suitable character set such as utf8mb4. A dedicated database user is then created with a strong password and only the privileges the application needs, rather than using the root account.

The third step is to create the database structure. The schema script from the project's database folder is run to create the tables, keys, constraints, indexes, views, triggers, and stored procedures. Initial data is then loaded, such as categories, an administrator account, and optionally sample books. The structure is verified with a few SQL queries in MySQL Workbench to confirm that every table exists and relationships are correct.

The fourth step is to configure the back end for production. The Spring Boot application.properties file, or a production profile, is updated with the database URL, username, and password, the JWT secret and expiry time, the allowed CORS origin, and the server port. Sensitive values are supplied through environment variables rather than being written into files that are stored in Git. Automatic schema modification by Hibernate is set to "validate" or "none" so that production tables are never altered unexpectedly.

The fifth step is to build the back end. Running the Maven command `mvn clean package` (or `gradle build`) compiles the code, runs the tests, and produces a single executable JAR file in the target folder. This JAR contains the application, its dependencies, and the embedded Tomcat server, so only a Java runtime is needed on the server. The file is then copied to the server using a tool such as SCP or SFTP.

The sixth step is to run the back end on the server. The server needs JDK 17 or later installed. The JAR is started with `java -jar library-app.jar`, and in production it is registered as a system service, for example with systemd on Linux, so that it starts automatically on reboot and restarts after a crash. The application logs are checked to confirm that it connected to the database and started on the configured port without errors.

The seventh step is to build the front end. The API base URL in the React environment file is changed from localhost to the address of the deployed back end. The command `npm install`

installs the dependencies and `npm run build` creates an optimised production build in the build (or dist) folder. This folder contains minified HTML, CSS, and JavaScript files that can be served by any web server or static hosting service.

The eighth step is to publish the front end. The contents of the build folder are uploaded to the chosen hosting platform, or copied to the web root of a server running Nginx. For a single-page React application, the server must be configured to return index.html for unknown paths, so that routes such as /my-books still work when the page is refreshed. After upload, the site is opened in a browser to confirm that it loads.

The ninth step is to secure and connect the system. A domain name is pointed to the server, an SSL certificate is installed to enable HTTPS, and a reverse proxy such as Nginx forwards API requests to the Spring Boot application. The firewall is configured to open only the necessary ports (80 and 443), and the MySQL port is restricted so it is not reachable from the internet. CORS settings are checked to make sure the deployed front-end address is allowed.

The final step is verification and maintenance. A smoke test is carried out on the live system: registering a member, logging in, searching, reserving a book, and viewing fines, along with an admin login. The Postman collection can be run against the production URL to check the endpoints. A backup schedule for the database is set up, logs are monitored, and the previous JAR and build are kept so that the system can be rolled back quickly if a problem appears.

9.2 Environment Configuration

Environment configuration means setting up the values and conditions under which the application runs in each environment. The project uses at least two environments: a development environment on the developer's computer and a production environment on the server. Each needs different settings, such as database addresses and passwords, and keeping them separate prevents accidents like test code touching real data.

The central principle is that configuration should be kept outside the source code. Settings such as database credentials, secret keys, and server addresses change from one environment to another and must never be hard-coded. Storing them in configuration files and environment variables lets the same compiled application run anywhere simply by supplying different values, and it keeps secrets out of the Git repository.

On the back end, Spring Boot uses application.properties (or application.yml) as its main configuration file. Environment-specific files such as application-dev.properties and application-prod.properties are activated using Spring profiles, selected with the property `spring.profiles.active`. The development profile points to a local database and enables detailed logging, while the production profile points to the real database and reduces logging to warnings and errors.

The database settings are the most important part of the configuration. They include `spring.datasource.url` (for example, jdbc:mysql://localhost:3306/librarydb), the username, the password, and the driver class. Hibernate settings such as the SQL dialect and the `ddl-auto` mode are also defined here. In development, ddl-auto can be "update" for convenience, whereas

in production it should be "validate" or "none" so that the schema is controlled only through the SQL scripts.

Security-related values form another group. The JWT secret key must be long, random, and different in every environment, and the token expiry time is set according to the required session length. The BCrypt strength and the CORS allowed origins are also configured. In production, the allowed origin is restricted to the real front-end domain, and the secret is supplied through an environment variable such as `JWT_SECRET` rather than being written in the file.

Environment variables are referenced inside the properties file with placeholders, for example `spring.datasource.password=${DB_PASSWORD}`. The actual values are defined on the server using the operating system, the systemd service file, a Docker configuration, or the settings page of a hosting platform. This way the repository contains only the names of the variables, never their values, which protects passwords even if the code is shared publicly.

The server environment itself must be prepared. The back-end machine needs a compatible Java runtime (JDK 17 or later), sufficient memory for the JVM, and the correct time zone setting, since loan due dates and fines depend on accurate dates. Java options such as maximum heap size can be set with flags like `-Xmx512m`. The application port, typically 8080, is set with `server.port`, and the file upload or request size limits can also be configured if needed.

The front-end configuration is handled with environment files. In a Vite project these are files such as .env.development and .env.production containing variables prefixed with VITE_ (or REACT_APP_ in Create React App), for example `VITE_API_BASE_URL=https://api.example.com`. The React code reads this value when creating the Axios instance. Because front-end variables are embedded into the public JavaScript during the build, they must never contain secrets, only public settings such as the API address.

The database environment requires its own configuration. This includes the MySQL character set and collation (utf8mb4), the time zone, the maximum number of connections, and the dedicated application user with limited privileges. The connection pool settings of HikariCP, such as the maximum pool size and timeouts, are tuned to match the expected load. For security, the database is configured to accept connections only from the application server, not from the public internet.

Finally, good practice around configuration is observed throughout. A `.env.example` or documentation file lists every required variable without real values, so a new developer or examiner can set up the project quickly. The `.gitignore` file excludes real .env files and any properties files containing secrets. Configuration is checked at startup, so the application fails clearly if a required value is missing, and every change in the production settings is recorded so that problems can be traced and corrected easily.

9.3 Hosting of Frontend and Backend

Hosting means placing the application on computers that are always switched on and connected to the internet so that members can reach it at any time. In this project, the three layers are hosted separately: the React front end as static files, the Spring Boot back end as a running Java service, and the MySQL database on a protected database

server. Hosting them separately lets each layer be managed and scaled independently.

The front end is hosted as a static site. After the build, it consists only of HTML, CSS, JavaScript, and image files, which need no server-side processing. Static hosting services such as Netlify, Vercel, GitHub Pages, or Amazon S3 with CloudFront are well suited to this. They are inexpensive or free for small projects, provide HTTPS automatically, and serve files quickly from servers around the world through a content delivery network (CDN).

Deploying the front end to a service like Netlify or Vercel is simple. The GitHub repository is connected to the platform, the build command (npm run build) and the output folder (build or dist) are specified, and the environment variable for the API URL is entered in the platform's settings. Each time new code is pushed to the main branch, the platform rebuilds and publishes the site automatically. A redirect rule sends all unknown paths to index.html so that React Router works after a page refresh.

The back end needs a platform that can run a Java application continuously. Options include a virtual machine such as an AWS EC2 instance or a DigitalOcean droplet, a platform-as-a-service such as Render, Railway, or Heroku-style services, or a container service that runs a Docker image. For a student project, a platform like Render or Railway is convenient because it builds and runs the application directly from GitHub with little server administration.

On a virtual machine, the back end is hosted more manually. After installing the JDK, the JAR is copied to the server and run as a systemd service so that it starts at boot and restarts after failures. Nginx is installed as a reverse proxy in front of the application, listening on ports 80 and 443 and forwarding requests to port 8080. This arrangement hides the internal port, allows HTTPS to be handled in one place, and can serve several applications on the same machine.

Docker offers a more portable way to host the back end. A Dockerfile describes the base Java image, copies the JAR into it, and defines the start command. The built image can be pushed to a registry and run on any server or cloud container service with the same behaviour as on the developer's machine. Docker Compose can start the Spring Boot container and a MySQL container together, which is also convenient for local demonstrations.

The database can be hosted in two main ways. It may be installed on the same server as the back end, which is the cheapest option and suitable for a prototype, or it may be a managed cloud database such as Amazon RDS, Google Cloud SQL, or a hosted MySQL service. Managed databases handle backups, updates, and failover automatically. In both cases, the database should accept connections only from the back end and should not be exposed to the public internet.

Communication between the hosted parts must be configured carefully. The front end calls the back end through its public HTTPS address, so the API base URL in the front-end build must match it exactly. The back end must list the front-end domain in its CORS settings, otherwise browsers will block the requests. Using subdomains, such as library.example.com for the front end and api.example.com for the back end, keeps the structure clear and makes

certificates easy to manage.

Security and reliability are essential in hosting. All traffic uses HTTPS with certificates from a provider such as Let's Encrypt, firewalls allow only the required ports, and secrets are stored as environment variables on the host. Regular database backups, log monitoring, and uptime checks help detect and recover from problems quickly. Free hosting tiers may put the server to sleep when idle, which can cause a slow first response, so this limitation should be noted when the system is demonstrated.

Finally, hosting decisions depend on the purpose and scale of the deployment. For an academic demonstration, a combination of a free static host for the React app, a free or low-cost platform for the Spring Boot service, and a small managed MySQL database is usually enough. For a real library with many members, the same architecture can be moved to larger cloud instances with load balancing, automated scaling, and monitoring, without changing the application design. This flexibility is a direct benefit of the layered, separately hosted architecture.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

During the development of **LibraryHub**, one of the initial challenges was connecting the Spring Boot backend with the MySQL database. Database connection errors and schema-related issues occurred when the application could not properly access the required tables or fields. These issues affected the startup of the backend and required checking the database name, username, password, and application configuration.

Another issue was related to **user authentication and login**. The frontend and backend needed to exchange the login response in the correct format. At one stage, the frontend expected the complete login response containing the token and user details, while the login component passed only the user object. This caused problems with storing the JWT token and identifying the logged-in user's role.

The project also faced **frontend and backend integration issues**. REST API requests sometimes returned errors such as HTTP 403 or 500. Authentication headers, JWT tokens, CORS configuration, and API endpoints had to be checked to determine whether the request was reaching the backend correctly. These issues showed the importance of testing the complete communication between the React frontend and Spring Boot backend.

Another challenge occurred during **database schema changes**. The User entity was modified to use a separate username and email instead of the previous name field. However, the database and Java entity were temporarily inconsistent, resulting in errors such as an unknown `name` column. Similar schema issues occurred with the notification `date` field when the database required a value that was not being supplied by the application.

The project also experienced some **frontend implementation and dependency issues**. JSX syntax errors occurred because of incorrectly closed HTML elements, while npm installation problems affected frontend dependencies. Features such as favourites, borrowing, reservations, and notifications also required careful handling of user IDs, book IDs, API responses, and empty responses. These issues required repeated testing and debugging before the modules could work together correctly.

10.2 Solutions Applied

The database connection issues were addressed by checking the **MySQL service, database configuration, credentials, and application properties**. The libraryhub database was configured as the application's database, and Hibernate was used to maintain the required tables. Database structure and entity definitions were also checked whenever a schema-related error occurred.

The authentication problem was solved by making the frontend and backend use a consistent

response structure. The login response contains the JWT token and user information, and the frontend stores both in localStorage. The login process was also updated so that the complete response is passed to the application, allowing the user's role and authentication information to be handled correctly.

For frontend-backend communication problems, the API service was improved to handle **JWT authorization, HTTP errors, empty responses, and invalid JSON responses**. The frontend attaches the stored token to protected requests using the Authorization: Bearer header. Browser developer tools and backend logs were used to inspect requests and responses and identify problems with authentication and API communication.

Database schema problems were handled by comparing the Java entity classes with the actual MySQL tables. Unwanted fields such as the old name column were removed from the intended design, while the new username field was introduced. Notification-related issues were addressed by ensuring that the required date value is generated by the application or database when a notification is created.

Frontend errors were resolved by checking component structure, props, API calls, and dependencies. JSX components were corrected when closing tags were missing, and callback functions for borrowing and reservations were properly passed from the main application to the catalog and modal components. ID comparisons were also made consistent by converting IDs to strings where necessary, improving the reliability of favourites and other user-specific features.

10.3 Current Limitations or Known Bugs

The current LibraryHub system mainly focuses on **digital library self-service** and does not completely automate physical library operations. Activities such as physically issuing books, scanning physical copies, arranging shelves, and checking returned physical books still require library staff. The application manages the digital records associated with these activities rather than replacing the complete physical workflow.

The system currently does not include a complete **online fine payment system**. Although borrowing and reservation information can be maintained, payment processing for overdue fines

is outside the present implementation. Automatic payment gateway integration and detailed fine calculation can be added as future enhancements.

Another limitation is that the current application is primarily designed as a **web-based system**. A dedicated Android or iOS mobile application is not part of the current implementation. Users can access the web interface through a browser, but native mobile features such as push notifications, device-specific functionality, and offline mobile access would require additional development.

The recommendation functionality is also limited in the current version. Books can be searched and filtered using available information, and users can maintain favourites, but the system does not currently provide advanced **AI-based personalized recommendations**. Future versions could analyze borrowing history, favourites, categories, and user preferences to generate personalized book suggestions.

Finally, the project may require additional testing before being used as a large-scale production library system. Performance testing with a very large number of users and books, advanced security testing, automated backups, and extensive deployment monitoring are not the main focus of the current academic implementation. These areas can be improved in future versions to make the system more suitable for large institutional libraries.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

LibraryHub can be enhanced with a **QR code-based book issue and return system**. Each book can have a unique QR code that can be scanned during borrowing and returning. This can reduce manual entry and make transactions faster.

A **due-date reminder system** can be added to notify users before their borrowed books are due. Notifications can also be generated for overdue books, upcoming reservations, and changes in reservation status.

The system can include an **online fine management feature**. Overdue fines can be calculated automatically based on the return date and users can view their outstanding fines through their accounts.

A **mobile application** can be developed for Android and iOS. Members would be able to search books, manage favourites, borrow or reserve books, and receive notifications directly through their mobile devices.

An **AI-based book recommendation feature** can also be introduced. The system could use a user's favourite books, borrowing history, and preferred categories to provide personalized book suggestions

11.2 Possible Integrations or Optimisations

LibraryHub can be integrated with **email and push notification services** to send automatic reminders about due dates, reservations, and important library updates. This would improve communication between the system and its users.

The application can be optimized by improving **database queries and indexing**. Frequently searched fields such as book title, author, ISBN, username, and email can be indexed to improve search speed as the number of records increases.

Caching can also be introduced for frequently accessed information such as book categories and popular books. This can reduce unnecessary database requests and improve the response time of

the application.

The system can be integrated with **cloud storage and hosting services** for easier deployment and scalability. Database backup and recovery mechanisms can also be introduced to protect important library records.

Finally, the application can be optimized through **security and performance testing**. API authentication, input validation, error handling, and access control can be strengthened, while load testing can be used to evaluate the system when many users access it simultaneously.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

LibraryHub is a full-stack library self-service application developed to simplify common library activities through a single digital platform. The system allows members to register and log in securely, search and browse books, check availability, manage favourites, borrow books, reserve unavailable books, and receive notifications. Administrators can manage library-related records and activities. The application uses React for the frontend, Spring Boot and REST APIs for the backend, and MySQL for database management.

12.2 What Was Achieved

The project successfully developed a centralized library management and self-service system with separate member and administrator functionalities. Key features such as user authentication, book catalog, search and filtering, favourites, borrowing, reservations, and notifications were implemented. The system connects the React frontend with the Spring Boot backend through REST APIs and stores application data in MySQL. These features help reduce manual work and provide easier access to library services.

12.3 Skills Learned During Development

During the development of LibraryHub, practical skills were gained in **React, HTML, CSS, JavaScript, Spring Boot, Java, REST APIs, and MySQL**. The project also provided experience with JWT authentication, role-based access, database design, CRUD operations, API testing using Postman, and frontend-backend integration. Additional skills were developed in debugging, error handling, Git and GitHub, software testing, and maintaining a full-stack application.

CHAPTER -13 REFERENCES

Books

Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.

Walls, C. (2022). *Spring in Action* (6th ed.). Manning Publications.

Tutorials and Learning Resources

W3Schools. (2025). *HTML, CSS, JavaScript, and SQL Tutorials.* Retrieved from https://www.w3schools.com

MDN Web Docs. (2025). *Web Development Documentation.* Retrieved from https://developer.mozilla.org

React. (2025). *React Documentation.* Retrieved from https://react.dev

GeeksforGeeks. (2025). *Java, React, Spring Boot, SQL, and Web Development Tutorials.* Retrieved from https://www.geeksforgeeks.org

freeCodeCamp. (2025). *Web Development and Programming Tutorials.* Retrieved from https://www.freecodecamp.org

Frameworks, APIs, and Tools Used

Spring. (2025). *Spring Boot Documentation.* Retrieved from https://spring.io/projects/spring-boot

Spring Security. (2025). *Spring Security Reference Documentation.* Retrieved from https://spring.io/projects/spring-security

MySQL. (2025). *MySQL 8.0 Reference Manual.* Retrieved from https://dev.mysql.com/doc/

Postman. (2025). *Postman Documentation.* Retrieved from https://learning.postman.com/docs/

GitHub. (2025). *GitHub Documentation.* Retrieved from https://docs.github.com

JSON Web Tokens. (2025). *Introduction to JSON Web Tokens.* Retrieved from https://jwt.io

Documentation and Research References

Oracle. (2025). *Java Documentation.* Retrieved from https://docs.oracle.com/en/java/

Jakarta Persistence. (2025). *Jakarta Persistence Documentation.* Retrieved from https://jakarta.ee/specifications/persistence/

Hibernate ORM. (2025). *Hibernate ORM Documentation.* Retrieved from https://hibernate.org/orm/documentation/

Roy Fielding. (2000). *Architectural Styles and the Design of Network-based Software Architectures.* University of California, Irvine.

OpenAPI Initiative. (2025). *OpenAPI Specification.* Retrieved from https://www.openapis.org

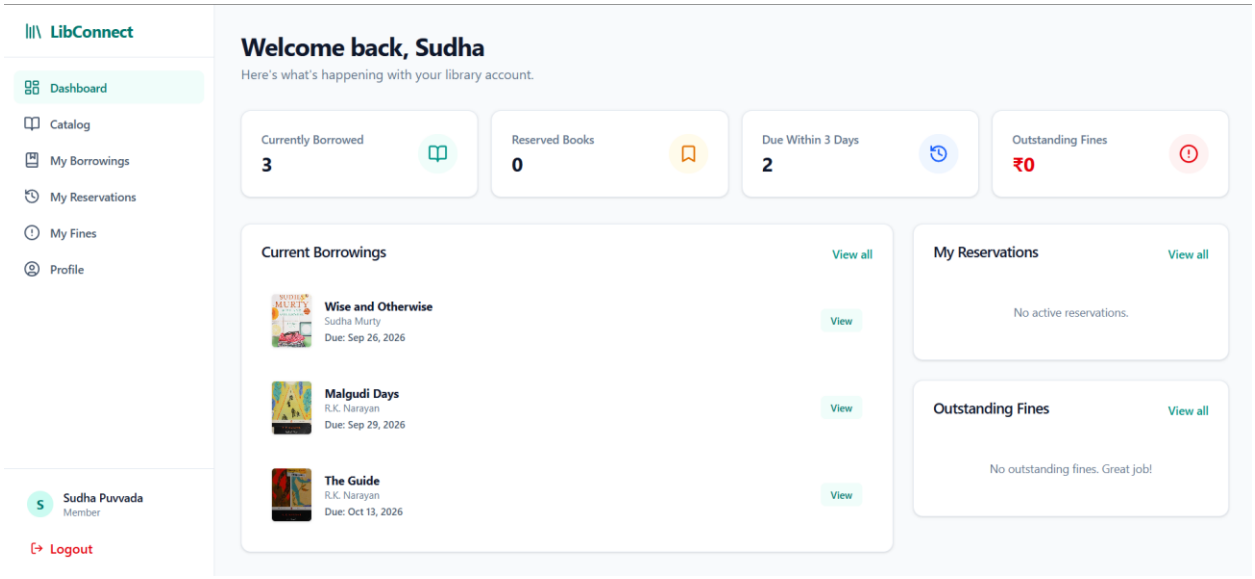
Project-Specific Resources

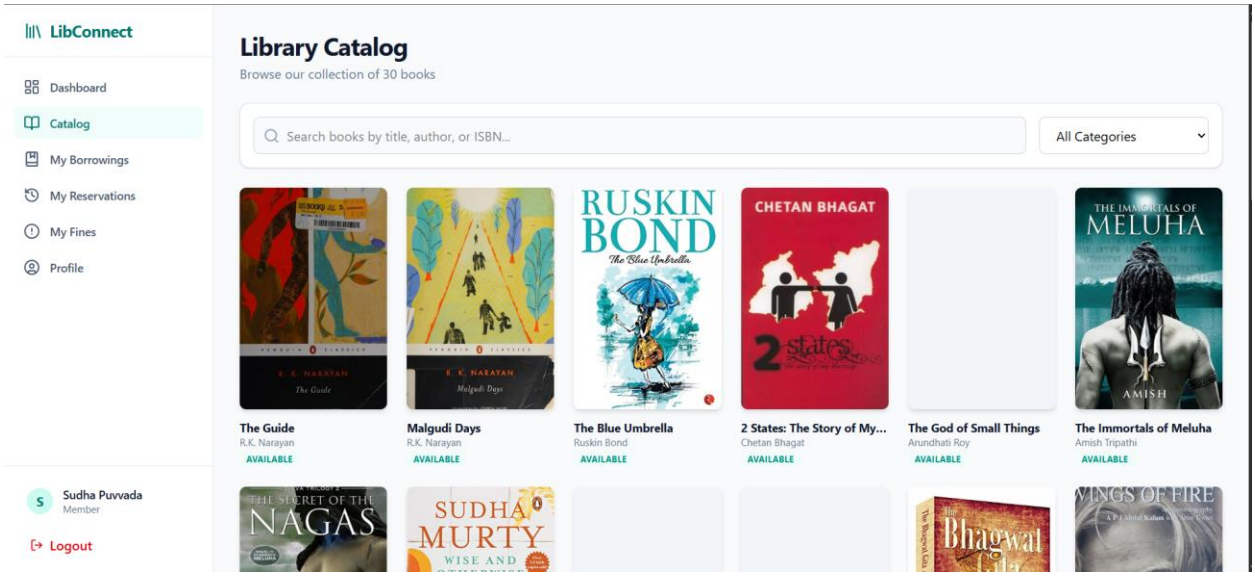
The LibraryHub project source code, database schema, REST API implementation, frontend components, authentication modules, and testing resources developed as part of the project were used as primary project resources.

The project also used VS Code, Git/GitHub, Postman, MySQL, Spring Boot, Spring Security, React, and JWT-based authentication during development and testing.

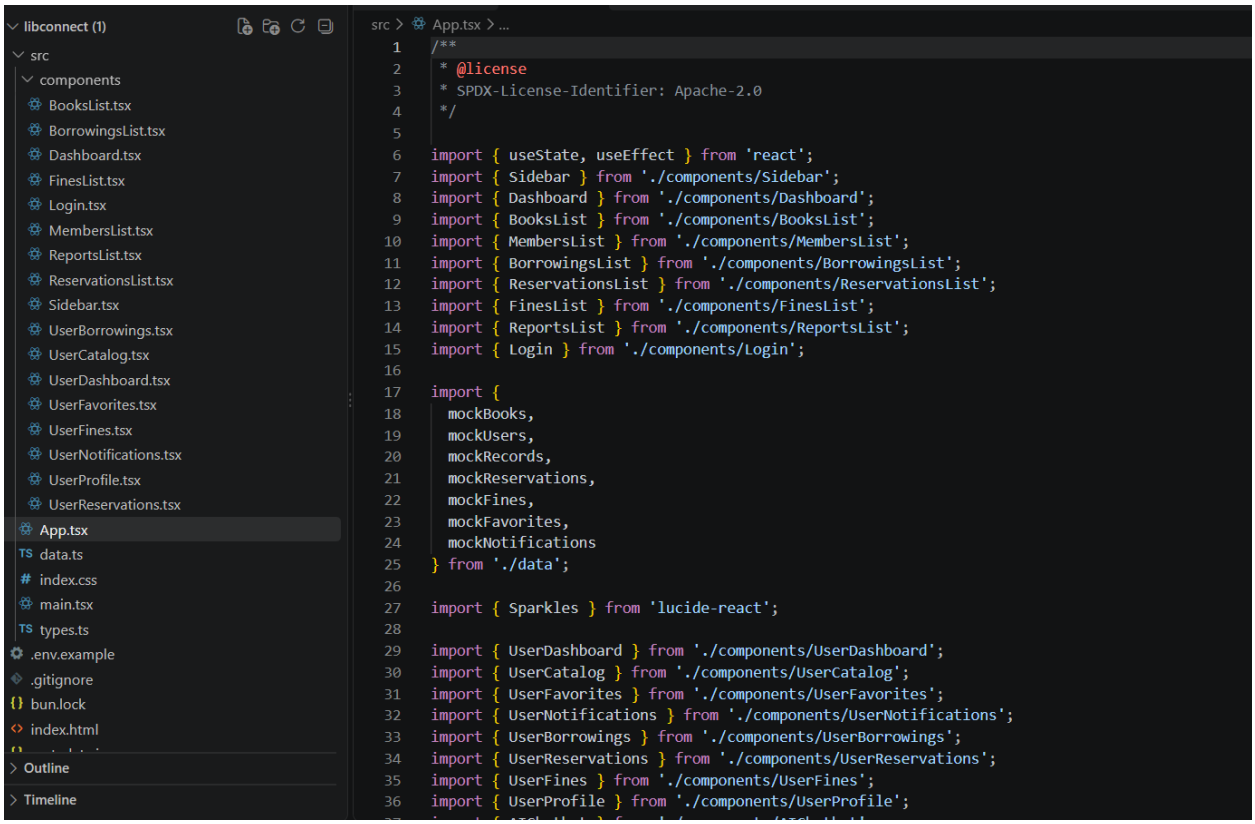
CHAPTER -14 APPENDICES

Screenshots of the app





Sample code snippets



```
libconnect (1)
src
components
BooksList.tsx
BorrowingsList.tsx
Dashboard.tsx
FinesList.tsx
Login.tsx
MembersList.tsx
ReportsList.tsx
ReservationsList.tsx
Sidebar.tsx
UserBorrowings.tsx
UserCatalog.tsx
UserDashboard.tsx
UserFavorites.tsx
UserFines.tsx
UserNotifications.tsx
UserProfile.tsx
UserReservations.tsx
App.tsx
data.ts
index.css
main.tsx
types.ts
.env.example
.gitignore
bun.lock
index.html
Outline
Timeline
```

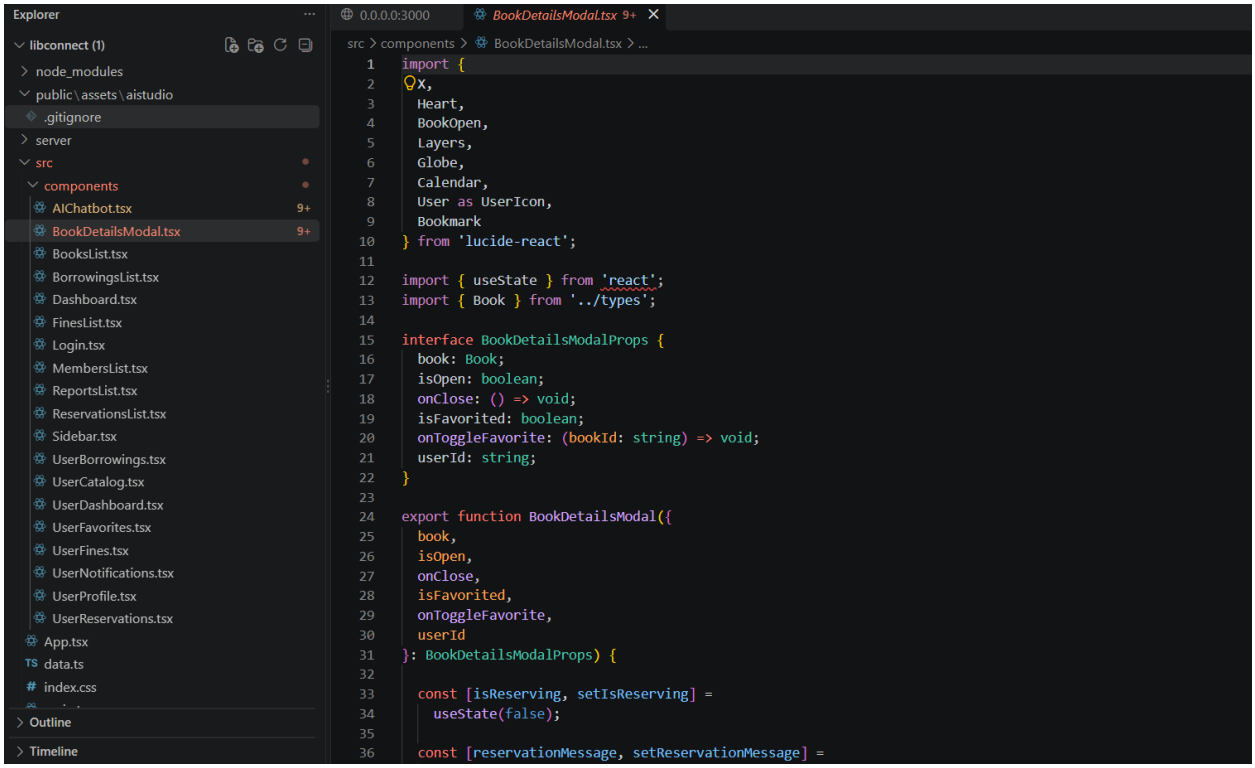
```
src > App.tsx > ...
/**
 * @license
 * SPDX-License-Identifier: Apache-2.0
 */

import { useState, useEffect } from 'react';
import { Sidebar } from './components/Sidebar';
import { Dashboard } from './components/Dashboard';
import { BooksList } from './components/BooksList';
import { MembersList } from './components/MembersList';
import { BorrowingsList } from './components/BorrowingsList';
import { ReservationsList } from './components/ReservationsList';
import { FinesList } from './components/FinesList';
import { ReportsList } from './components/ReportsList';
import { Login } from './components/Login';

import {
  mockBooks,
  mockUsers,
  mockRecords,
  mockReservations,
  mockFines,
  mockFavorites,
  mockNotifications
} from './data';

import { Sparkles } from 'lucide-react';

import { UserDashboard } from './components/UserDashboard';
import { UserCatalog } from './components/UserCatalog';
import { UserFavorites } from './components/UserFavorites';
import { UserNotifications } from './components/UserNotifications';
import { UserBorrowings } from './components/UserBorrowings';
import { UserReservations } from './components/UserReservations';
import { UserFines } from './components/UserFines';
import { UserProfile } from './components/UserProfile';
```



```
Explorer
libconnect (1)
node_modules
public\assets\aistudio
.gitignore
server
src
components
AIChatbot.tsx 9+
BookDetailsModal.tsx 9+
BooksList.tsx
BorrowingsList.tsx
Dashboard.tsx
FinesList.tsx
Login.tsx
MembersList.tsx
ReportsList.tsx
ReservationsList.tsx
Sidebar.tsx
UserBorrowings.tsx
UserCatalog.tsx
UserDashboard.tsx
UserFavorites.tsx
UserFines.tsx
UserNotifications.tsx
UserProfile.tsx
UserReservations.tsx
App.tsx
data.ts
index.css
Outline
Timeline
```

```
0.0.0.0:3000
BookDetailsModal.tsx 9+
src > components > BookDetailsModal.tsx > ...
import {
X,
  Heart,
  BookOpen,
  Layers,
  Globe,
  Calendar,
  User as UserIcon,
  Bookmark
} from 'lucide-react';

import { useState } from 'react';
import { Book } from '../types';

interface BookDetailsModalProps {
  book: Book;
  isOpen: boolean;
  onClose: () => void;
  isFavorited: boolean;
  onToggleFavorite: (bookId: string) => void;
  userId: string;
}

export function BookDetailsModal({
  book,
  isOpen,
  onClose,
  isFavorited,
  onToggleFavorite,
  userId
}: BookDetailsModalProps) {

  const [isReserving, setIsReserving] =
    useState(false);

  const [reservationMessage, setReservationMessage] =
```

Installation/setup instructions

The LibraryHub application consists of a React-based frontend, Spring Boot backend, and MySQL database. The system requires the frontend and backend environments to be configured separately so that the user interface can communicate with the REST APIs and database.

First, copy or extract the complete LibraryHub project folder to the local computer. The project should contain the frontend and backend source-code folders. Open the project using **Visual Studio Code** or another suitable development environment. Make sure that Java, Node.js, npm, Maven, and MySQL are installed and properly configured before starting the application.

Next, configure the MySQL database for the application. Create the required LibraryHub database and configure the database connection details in the Spring Boot backend's application.properties file. The username, password, database name, and server port should match the local MySQL configuration. The Spring Boot application uses JPA/Hibernate to communicate with the database and manage the required tables.

After configuring the database, open a terminal in the backend directory and start the Spring Boot application using Maven. The backend runs on the configured server port and provides REST API endpoints for authentication, books, borrowing, reservations, favourites, notifications, and other library operations.

Finally, open a separate terminal in the frontend directory and install the required npm dependencies. Start the React development server using the appropriate npm command. The LibraryHub frontend will then be available through the local development address, normally http://localhost:5173, while the backend API runs on http://localhost:8080. Once both services are running, the application can be accessed through a modern web browser such as Google Chrome, Microsoft Edge, or Firefox.

User Manual or Guide

When the LibraryHub application is opened, the user is presented with the main interface and authentication options. New users can select **Register** and provide their username, email address, and password. Existing users can select **Login** and sign in using their registered email or username and password.

After successful authentication, members can access the library catalog and browse the available books. The catalog provides information such as book title, author, ISBN, category, description, and availability. Users can search for books and apply available filtering options to find the required book more easily.

Members can also open individual book details and use features such as **Favourites**, **Borrow**, and **Reserve**, depending on the availability of the selected book. Books that are currently available can be borrowed, while unavailable books can be reserved when the reservation functionality is applicable. Users can also view their borrowing and reservation information.

The application provides notifications to keep users informed about relevant library activities. Members can manage their account-related information and access their library activities through the application. Authentication and role-based access ensure that members and administrators receive access to the functions intended for their respective roles.

Administrators have additional management functionality for maintaining library records. Depending on the implemented modules, administrators can manage books, users, borrowing records, reservations, and notifications. Overall, LibraryHub provides a centralized web-based interface through which members can perform common library self-service activities without depending entirely on manual library procedures.

Geo Tag photos with guide

